

# Sam's Portfolio

---

None

Sam Bell

None

# Table of contents

---

|                                            |           |
|--------------------------------------------|-----------|
| <b>1. Sam Bell's Portfolio of Projects</b> | <b>3</b>  |
| 1.1 About Me                               | 3         |
| 1.2 Recent Achievements                    | 3         |
| 1.3 My Favourite Projects                  | 4         |
| 1.4 My Skills                              | 4         |
| 1.5 Get in Touch                           | 4         |
| <b>2. My Projects</b>                      | <b>5</b>  |
| <b>3. Programming</b>                      | <b>14</b> |
| 3.1 Chess Bot                              | 14        |
| 3.2 20 Hour Project                        | 16        |
| 3.3 SamSpeak                               | 20        |
| 3.4 QR Code Generator                      | 23        |
| 3.5 Scripted Journeys                      | 26        |
| 3.6 Sudoku Solver                          | 29        |
| 3.7 Morse Code Translator                  | 36        |
| <b>4. Electronics</b>                      | <b>39</b> |
| 4.1 CPU                                    | 39        |
| 4.2 Safe                                   | 47        |
| <b>5. Digital Making</b>                   | <b>50</b> |
| 5.1 Home Server                            | 50        |
| 5.2 Laptop                                 | 56        |
| 5.3 Light Fantastic                        | 58        |
| 5.4 Magic Mirror                           | 61        |
| 5.5 Wildlife Camera                        | 63        |
| <b>6. Robotics</b>                         | <b>64</b> |
| 6.1 World Educational Robotics 2025        | 64        |
| 6.2 Picar S                                | 76        |
| 6.3 My First Robot                         | 77        |

# 1. Sam Bell's Portfolio of Projects

---



## 1.1 About Me

---

I am a 15 year old electronics and programming enthusiast. I love computer science and everything related to digital making, and have since I learnt the basics of Python when I was 8. Since then, it has become my main hobby and the direction I want my career to go in. In 2020, I got my first Raspberry Pi and learnt about electronics. Recently I realised that digital making and robotics combine both of my main interests, and those are what I'm focusing on now. I have made loads (82 that I still have records of!) of projects of varying degrees of difficulty. This portfolio contains the ones that I am most proud of.

---

## 1.2 Recent Achievements

---

- **British champion** and **3rd place worldwide** in an international robotics competition (WER 2025)
  - **Finalist** in Cyber Switchup 2023 (<https://cyberswitchup.net/>) (I missed this year's deadline)
  - Predicted **Grade 9** in GCSE Computer Science
-

## 1.3 My Favourite Projects

| Project           | Description                                                                         | Link                             |
|-------------------|-------------------------------------------------------------------------------------|----------------------------------|
| Chess Bot         | I created a fully-functional lookahead evaluation chess bot                         | <a href="#">View Project</a>     |
| QR Code Generator | I made a v5 QR code generator in python                                             | <a href="#">View Project</a>     |
| WER 2025          | An international robotics competition where I came 1st in the UK in my age category | <a href="#">More information</a> |

## 1.4 My Skills

### 1.4.1 Technical Skills

- **Programming:** I know how to program in many languages (e.g. C, HTML/CSS, and Go, to name a few), but my strongest are Python and Java.
- **Electronics:** Soldering, PCB Design (KiCad), and simply being able to work out how stuff should work.
- **Tools:** Git, Raspberry Pi, CAD (Autodesk Inventor & OpenSCAD)
- **Systems:** I understand how systems like Linux work due to me constantly changing, fixing and improving them on my own computer.

### 1.4.2 Soft Skills

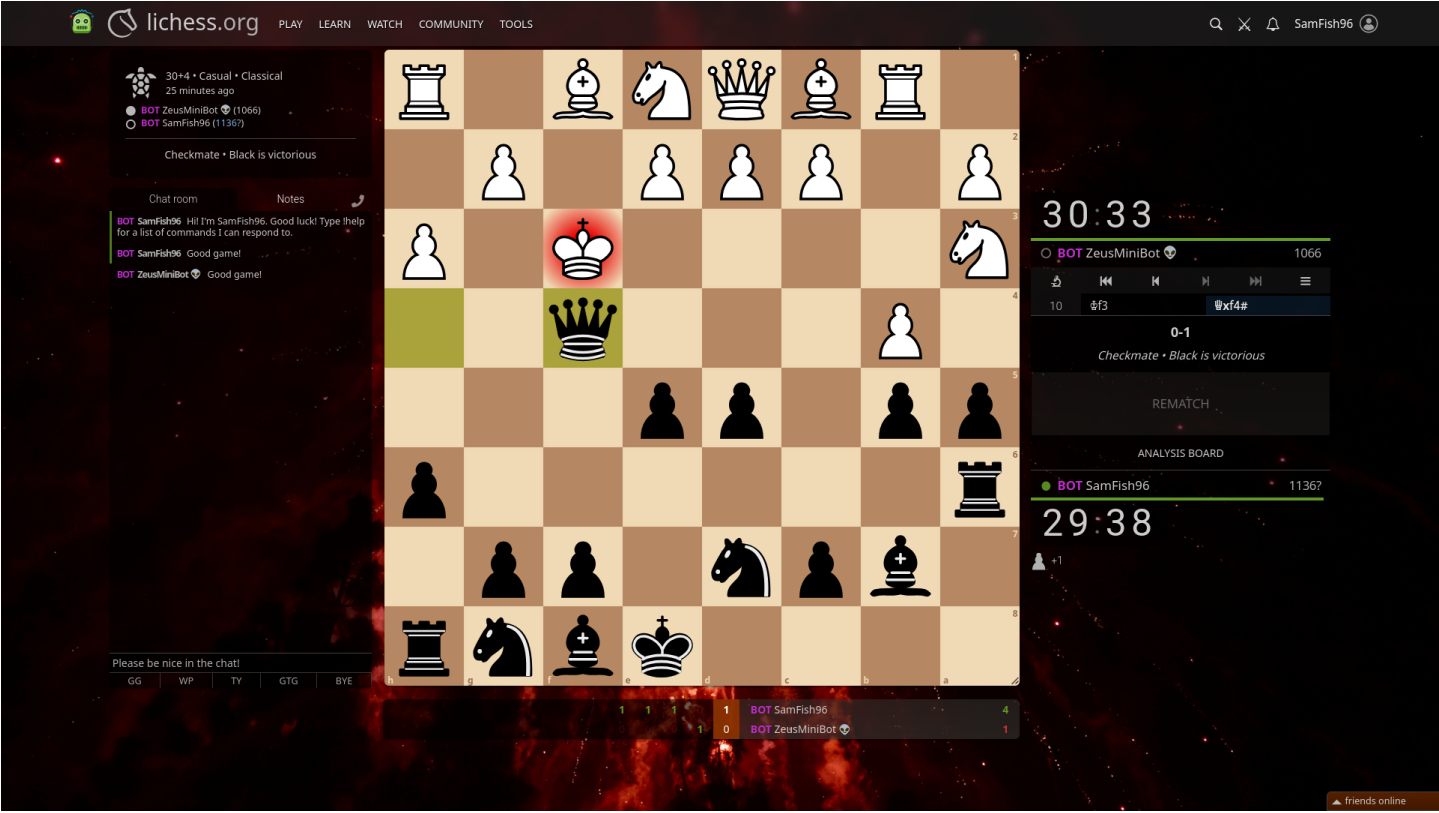
- **Problem Solving** - I have come across many challenges over my 5 years of making, and seeing as my family and friends aren't interested in coding or making, I have to solve everything myself using the internet, magazines and books. This has made me pretty good at problem solving.
- **Curiosity** - I have always wanted to know how things work, and I usually find this out by making it (e.g. my QR code generator)
- **Resilience** - I just refuse to give up, even if a problem takes me years to solve! I started many of my projects (such as the Light Fantastic) when I was younger and didn't have enough knowledge, skills or experience, but I kept coming back to them until I managed to finish them.

## 1.5 Get in Touch

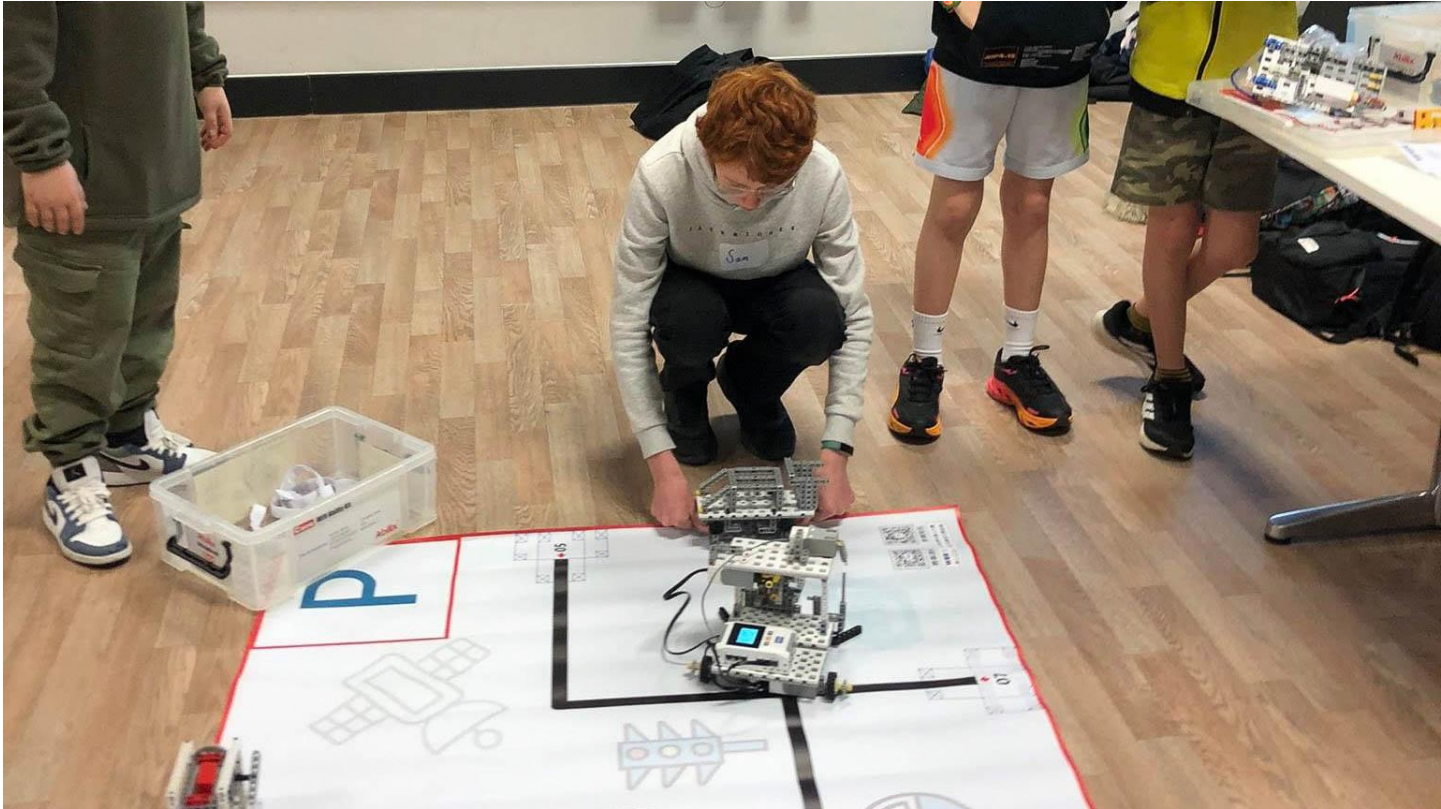
- **Email:** [samgbell2011@icloud.com](mailto:samgbell2011@icloud.com) (<mailto:samgbell2011@icloud.com>)
- **Phone:** 07468 779147
- **Portfolio:** <https://sambell2.github.io> (<https://sambell2.github.io>)

### 1.5.1 View My Full Portfolio →

# 2. My Projects



Chess Bot  
(2025 - Aged 14)



WER 2025  
(2025 - Aged 14)



20 Hour Project  
(2026 - Aged 15)

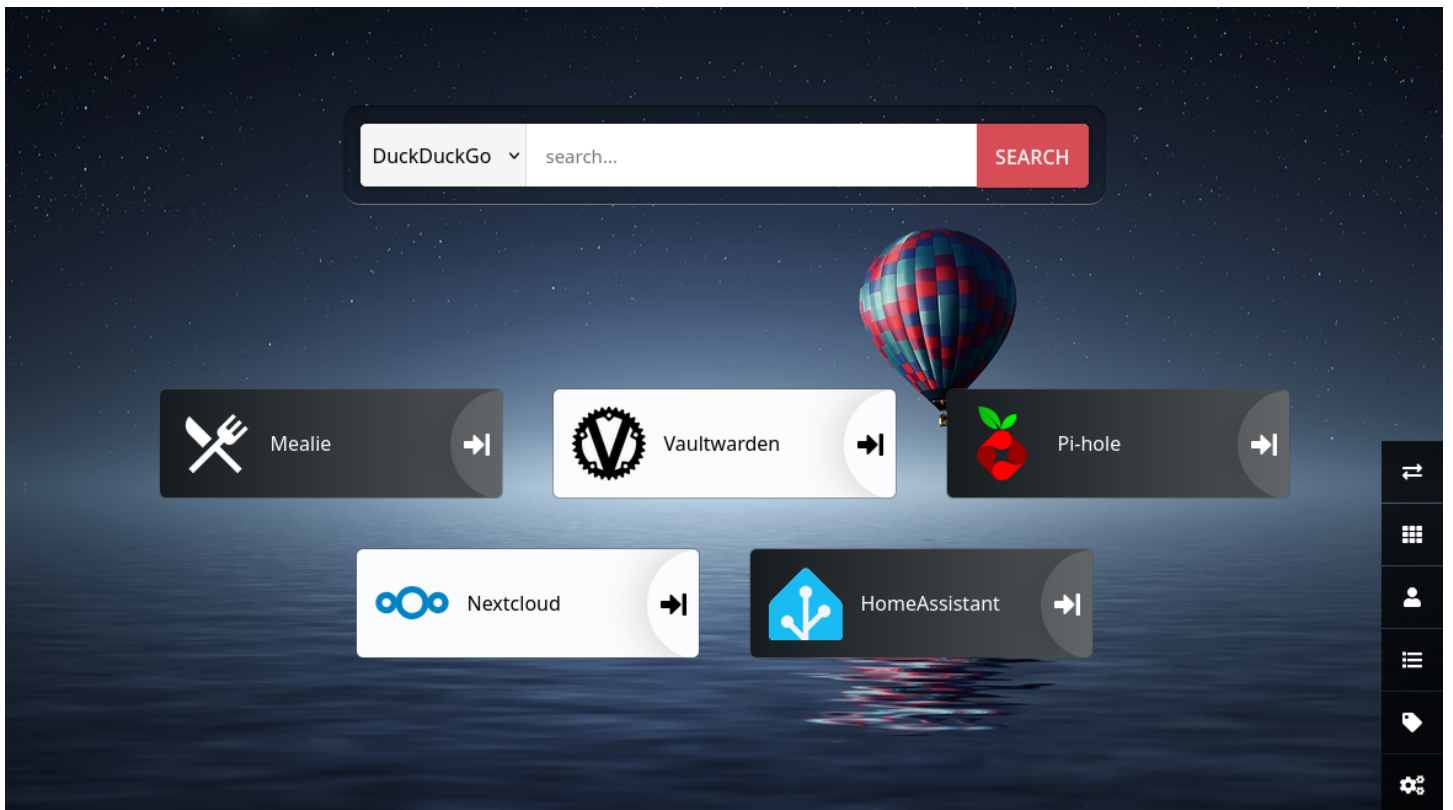
```

Preprocessor.py - SamSpeak - VSCode
1 import os
2
3 class Preprocessor:
4     def __init__(self, filename, SamSpeak, code):
5         self.filesRead = [filename]
6         self.SamSpeak = SamSpeak
7         self.code = code.split('\n')
8
9     def preprocess(self):
10        self.code = '\n'.join(self.code)
11        self.code = self.code.replace('\n', '\n')
12        self.code = self.code.split('\n')
13        for num, line in enumerate(self.code):
14            if line.startswith('#'):
15                self.code[num] = ""
16            if not os.path.exists(f"{line[1:]}"):
17                self.SamSpeak.scanError(num+1, "This file doesn't exist!")
18                continue
19            if f"{line[1:]}".ss in self.filesRead:
20                continue
21            else:
22                self.filesRead.append(f"{line[1:]}")
23            with open(f"{line[1:]}") as file:
24                lineNum = 0
25                for line in file.readlines():
26                    self.code.insert(num+lineNum, line.strip())
27                    lineNum += 1
28        self.code = self.preprocess()
29        elif line.startswith('!'):
30            self.code[num] = ""
31            self.SamSpeak.interpreter.addModule(line[1:])
32            self.code = self.preprocess()
33        return self.code
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2
```



## QR Code Generator

(2024 - Aged 13)



## Home Server

(2026 - Aged 15)



```

1
arco@alarmi-
You see an exit to the North.
You see an exit to the South.
You see an exit to the East.
> move north

Crystal Grotto
Crystals embedded in the walls glow with a soft, ethereal light, creating a mesmerizing display.
There are no items here.
A massive golem made of living crystal, slow but immensely strong.
You see an exit to the North.
You see an exit to the South.
You see an exit to the East.
> fight golem

A battle begins with the Golem!
You hit the Golem with your Fists. It causes 0.5 damage.
The Golem hits you with its Runic Hammer. It causes 3 damage.
HP: [██████████] 7/10 HP
Continue?
> yes
You hit the Golem with your Fists. It causes 0.5 damage.
The Golem hits you with its Runic Hammer. It causes 3 damage.
HP: [██████████] 4/10 HP
Continue?
> yes
You hit the Golem with your Fists. It causes 0.5 damage.
The Golem hits you with its Runic Hammer. It causes 3 damage.
HP: [██] 1/10 HP
Continue?
> 

```

### Scripted Journeys

(2024-2025 - Aged 13-14)



### Magic Mirror

(2023-2025 - Aged 12-14)

The screenshot shows a Notepad++ window with a Python script titled 'SudokuSolver.py'. The script implements a backtracking algorithm to solve a 9x9 Sudoku puzzle. It includes functions for finding the next empty cell, validating a guess, and solving the puzzle. A Tkinter window titled 'tk' is overlaid on the right, displaying a solved Sudoku grid. The grid is as follows:

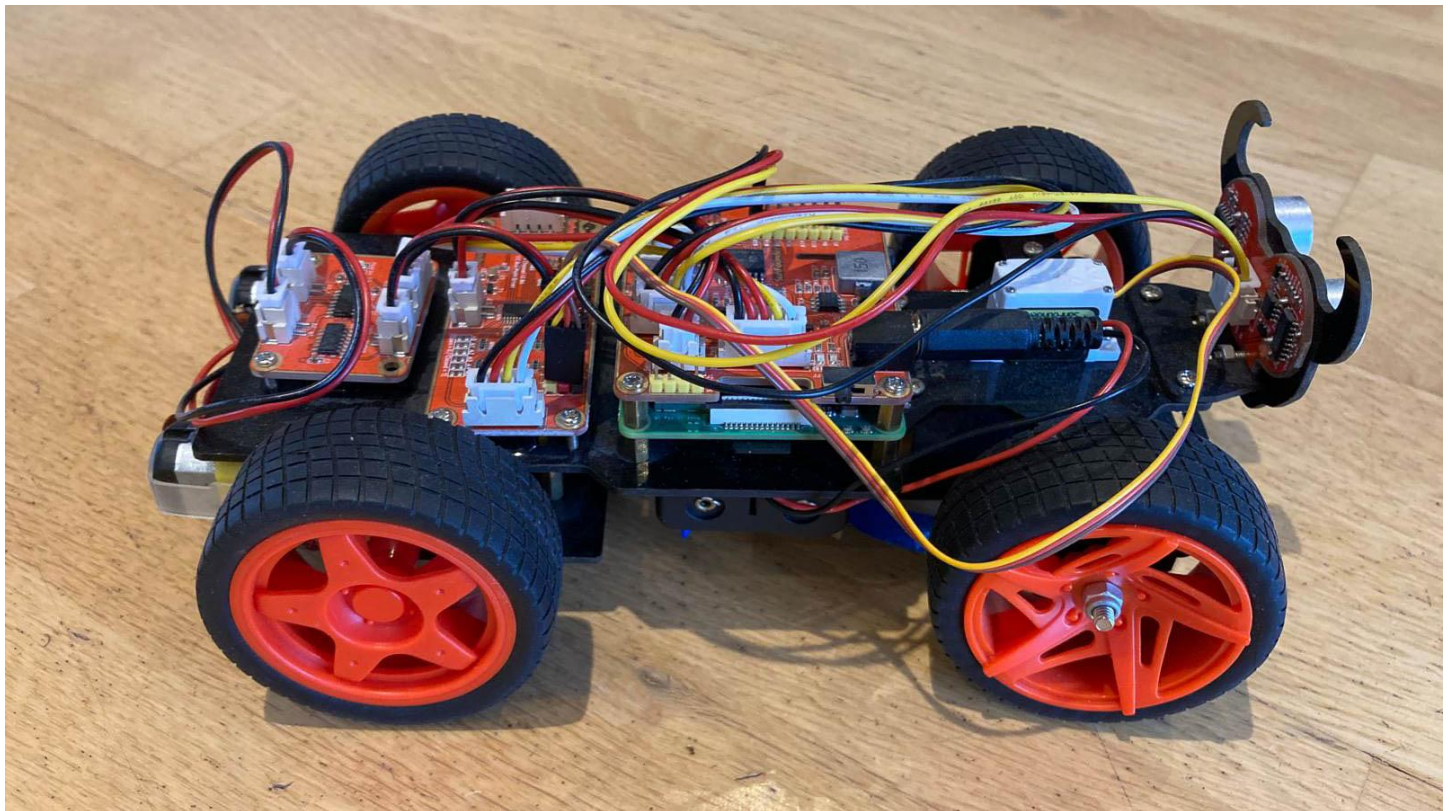
|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 9 | 8 | 2 | 3 | 4 | 6 | 7 |   |
| 2 | 3 | 4 | 1 | 6 | 7 | 5 | 8 | 9 |
| 5 | 6 | 7 | 4 | 8 | 9 | 1 | 2 | 3 |
| 3 | 1 | 2 | 5 | 4 | 6 | 7 | 9 | 8 |
| 4 | 5 | 6 | 7 | 9 | 8 | 2 | 3 | 1 |
| 7 | 8 | 9 | 3 | 1 | 2 | 6 | 4 | 5 |
| 6 | 2 | 1 | 8 | 3 | 5 | 9 | 7 | 4 |
| 8 | 7 | 5 | 9 | 2 | 4 | 3 | 1 | 6 |
| 9 | 4 | 3 | 6 | 7 | 1 | 8 | 5 | 2 |

Below the grid, the Tkinter window has a 'Complete' label and three buttons: 'Solve', 'Clear', and 'Hint'.

Python file length: 8,005 lines: 219 Ln: 27 Col: 35 Pos: 860 Windows (CR LF) UTF-8

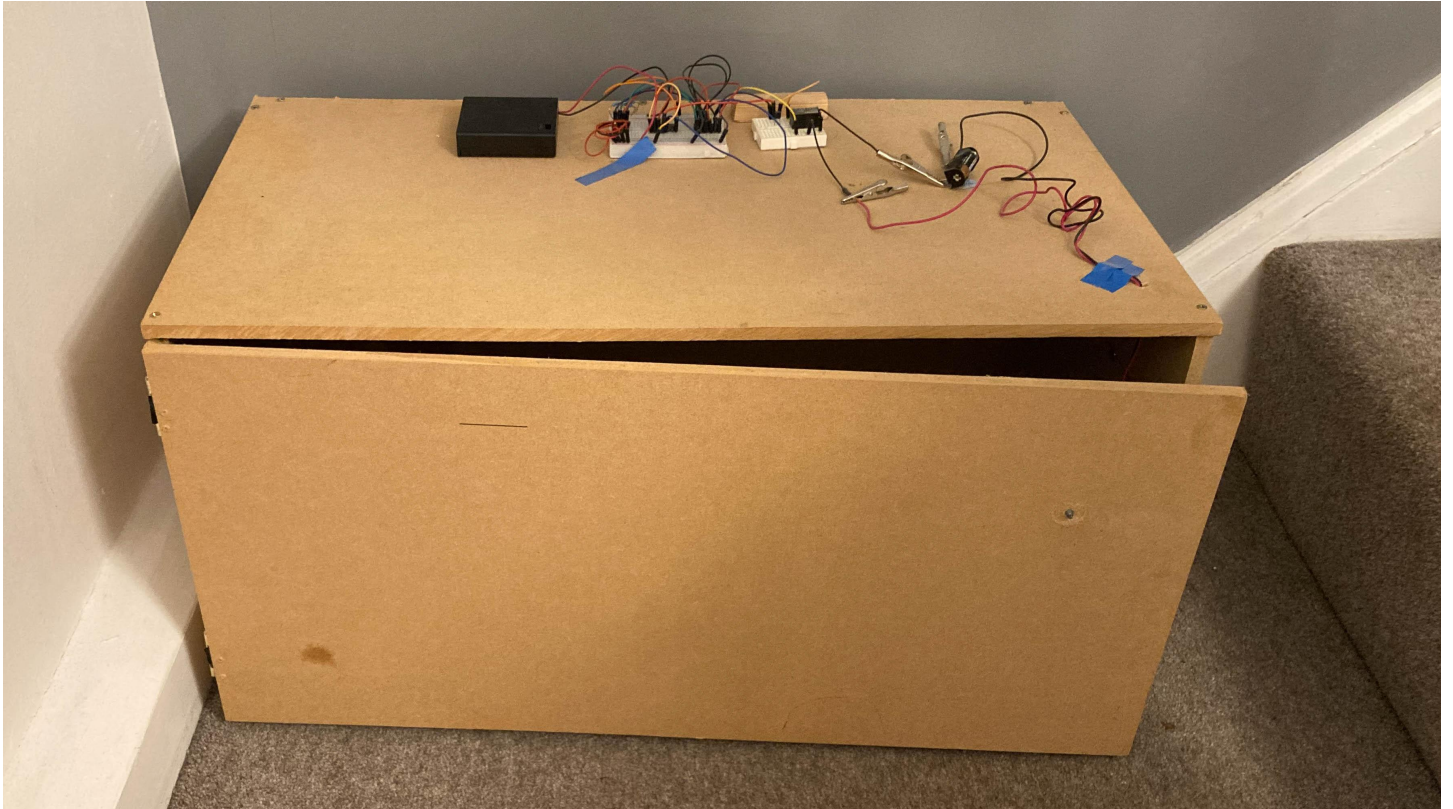
## Sudoku Solver

(2023 - Aged 12)



## Picar S

(2023-2025 - Aged 12-14)



Safe  
(2021 - Aged 10)



Wildlife Camera  
(2022 - Aged 11)

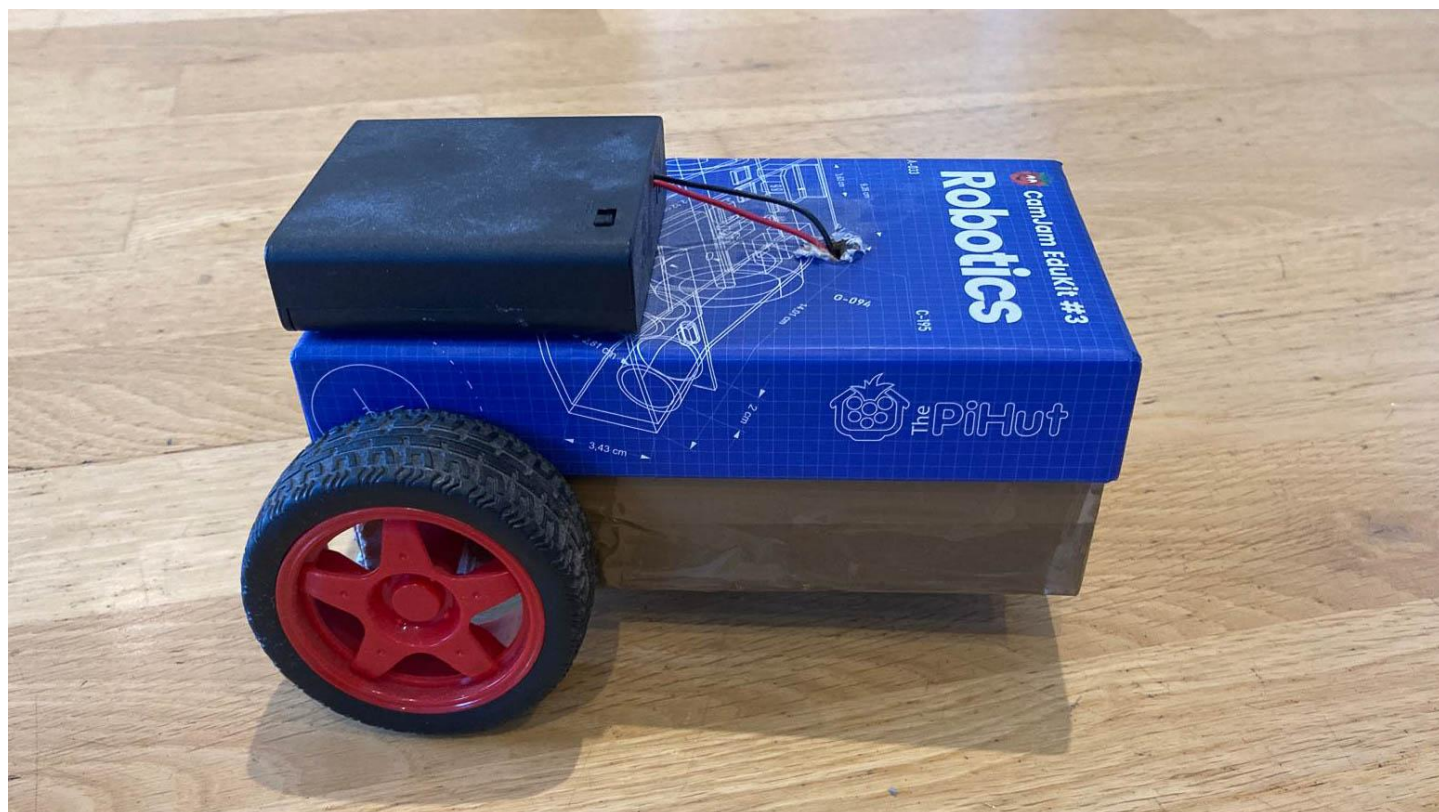
The image shows a VS Code editor window with the file `morse.py` open. The script defines a `Translator` class with a `fromMorse` method and a main block that takes user input and prints the translated message.

```
1 class Translator():
12     def fromMorse(self, msg):
19         for letter in letters_in_word:
20             try:
21                 message_in_english += letters[letter]
22             except:
23                 message_in_english += '#'
24         message_in_english += ' '
25         return message_in_english
26
27 if __name__ == "__main__":
28     translator = Translator()
29     msg = input("Enter message to translate to morse: ")
30     print(translator.toMorse(msg))
31     msg = input("Enter message in morse to translate to English: ")
32     print(translator.fromMorse(msg))
```

The terminal output shows the script being executed in a virtual environment. The user enters "hello world" for the first prompt and ".-.. .-.. --- .-- --- .-. .-.. ---" for the second prompt, resulting in the output "hello world".

```
[X] [arco@alarm] - [~/Documents/SamBell2.github.io]
• tensions/ms-python.debugpy-2025.14.1-linux-arm64/bundled/libs/debugpy/adapters/.../debugpy/launcher 57435 -- /
home/arco/Documents/Python/morse/morse.py
Enter message to translate to morse: hello world
.... .-.. .-.. --- .-- --- .-. .-.. ---
Enter message in morse to translate to English: .... .-.. .-.. --- .-- --- .-. .-.. ---
hello world
[arco@alarm] - [~/Documents/SamBell2.github.io]
$
```

## Morse Code Translator (2021 - Aged 10)

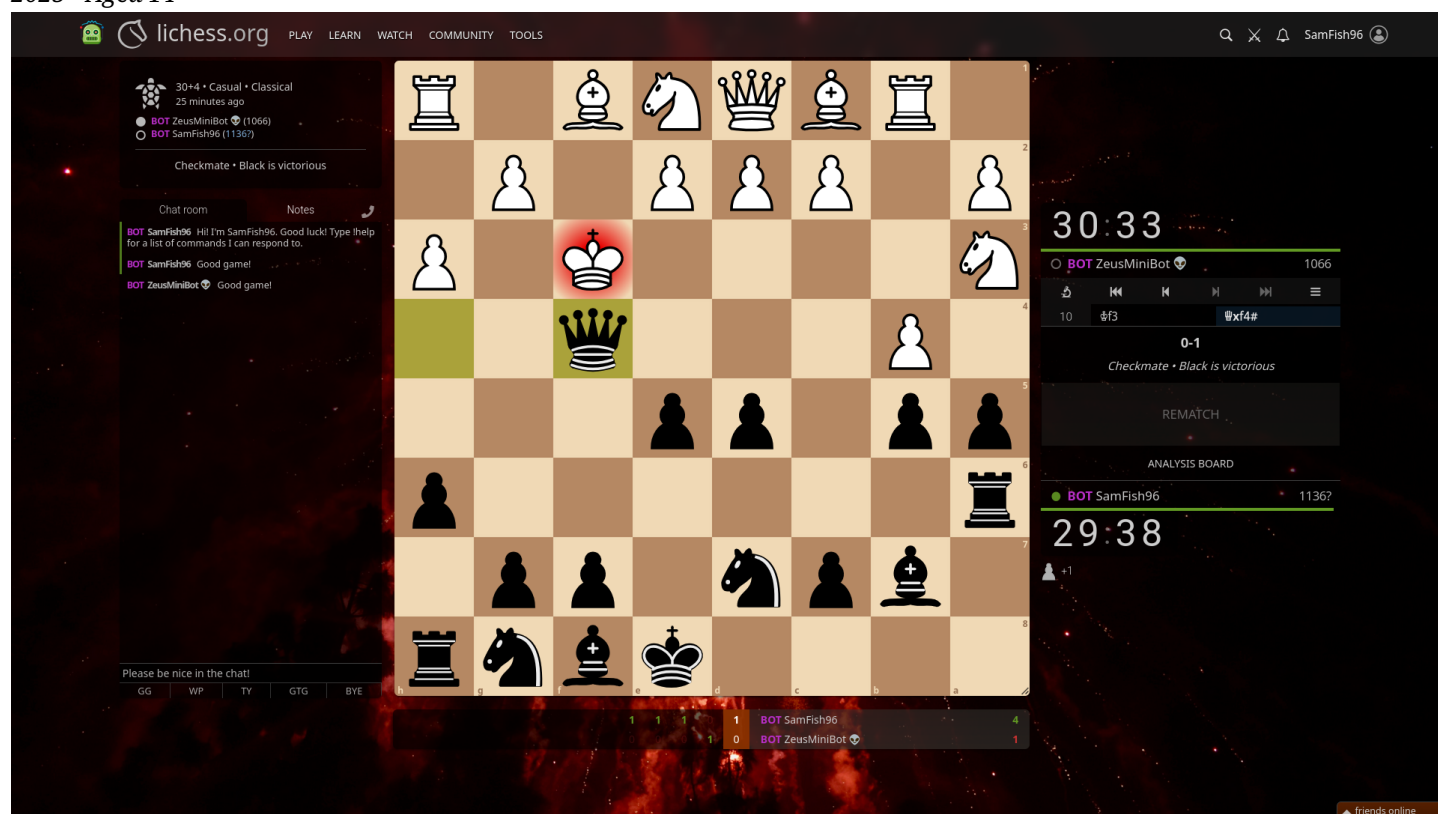


CamJam Robot  
(2022 - Aged 11)

# 3. Programming

## 3.1 Chess Bot

2025 - Aged 14



A Lichess.org (<https://lichess.org/h98ou7COBKNf>) game where my bot won

### 3.1.1 Overview

I created a Java program that can play chess. It generates all legal moves and evaluates them, giving them points. It then plays the highest-ranking move. I have given it a UCI (Universal Chess Interface) frontend so it can interact with other chess apps. For example, I have given it a Lichess account (SamFish96 (<https://lichess.org/@/SamFish96>)) and occasionally it is online. You can see the source code on GitHub (<https://github.com/SamBell2/SamFish/tree/master/src/chess>).

### 3.1.2 Inspiration

I've been interested in complex decision making programs for a while, and a chess bot was a suitably complex but not overwhelming introduction to them.

## 3.1.3 How it works

---

It uses an 8x8 array of pieces or empty squares to represent the board. Whenever the chess app tells it that a move has been made, it simulates the move on its own board. When it needs to move one of its pieces, it looks at all of the possible moves that it can make, discards any that put it in check, then evaluates all of them. If it has time, it looks at the next turn or two as well. It then picks the move it thinks is best and plays it.

---

## 3.1.4 Key Features

---

- Generates almost all possible moves.
  - Uses syzygy tablebases (<https://syzygy-tables.info/>) for the endgame to ensure perfect moves.
  - Detects checks, checkmates, stalemates and threefold repetition.
  - Automatic detailed logging to see the bot's logic.
  - Interacts with the Lichess bot API to play online.
- 

## 3.1.5 Improvements

---

- Add castling and en passant capabilities
  - Increase speed
- 

## 3.1.6 Challenges

---

I learnt a lot about Java, as before I had mainly programmed in Python. In particular, I use interfaces quite a bit. Looking for checks, checkmates, stalemates and other draws was far harder than I had anticipated, but I did eventually manage. Interacting with syzygy tables was a bit too complex for me to do (there are no Java libraries to read them) so I used an existing library called Fathom (<https://github.com/jdart1/fathom/>) to do it.

## 3.2 20 Hour Project

2026 - Aged 15



*Partway through a game*

### 3.2.1 Overview

For OCR GCSE computer science, everyone has to complete a project. There were 6 different briefs which I could pick from, then I had 20 hours in lessons to plan, program, test and complete the app. I chose a relatively simple brief, so that I could then put the vast majority of the time into extras that would be more interesting. I turned the simple game into a more complex online multiplayer game that was as secure as I could make it.

---

## 3.2.2 Brief

Katarina is developing a two-player dice game. The players roll two 6-sided dice each and get points depending on what they roll. There are 5 rounds in a game. In each round, each player rolls the two dice. The rules are:

- The points rolled on each player's dice are added to their score.
- If the total is an even number, an additional 10 points are added to their score.
- If the total is an odd number, 5 points are subtracted from their score.
- If they roll a double, they get to roll one extra die and get the number of points rolled added to their score.
- The score of a player cannot go below 0 at any point.
- The person with the highest score at the end of the 5 rounds wins.
- If both players have the same score at the end of the 5 rounds, they each roll 1 die and whoever gets the highest score wins (this repeats until someone wins).

Only authorised players can play the game. Where appropriate, input from the user should be validated.

Design, develop, test and evaluate a program that:

1. Allows two players to enter their details, which are then authenticated to ensure that they are authorised players.
2. Allows each player to roll two 6-sided dice.
3. Calculates and outputs the points for each round and each player's total score.
4. Allows the players to play 5 rounds.
5. If both players have the same score after 5 rounds, allows each player to roll 1 die each until someone wins.
6. Outputs who has won at the end of the 5 rounds.
7. Stores the winner's score, and their name, in an external file.
8. Displays the score and player name of the top 5 winning scores from the external file.

---

## 3.2.3 My Plan

I quickly decided to turn this into an online multiplayer game, instead of just 1 computer. This meant that I needed a client-server model, with 1 server that ran all the games and multiple clients, 1 for each player. In order to make this secure, I used TLS to encrypt the data between server and client. The brief also stated that the users were authenticated, which I spent a while securing. I used an encrypted server-side database to store usernames, and I stored the encrypted password hash instead of the password itself so that even if someone got an unencrypted dump of the DB they still wouldn't be able to log in with someone else's account. All of the game logic, such as dice rolls, were done server-side so that a modded client couldn't say "I rolled a 6 every time" and then win. No private information was in the code, because Python is an interpreted language so anyone could read the code.

---

## 3.2.4 TLS

I used TLS to ensure that no-one could sniff packets between the server and the clients. This meant that you couldn't capture packets and use those to find people's login information. This was done with the Python `ssl` module, and the actual transport handled by `socket`, meaning I could use my own protocol instead of using standard HTTP. The certificate I used was generated by `openssl` and was self-signed because I was using IP addresses and `.local` hostnames for testing and Let's Encrypt doesn't generate certificates for those.

---

## 3.2.5 Login

---

When the client first requests to login, it sends the username to the server, which then responds with the salt for that account. The salt is a random number stored with the password so that even if multiple accounts have the same password, they all have different hashes. The server also sends a random number (known as a "nonce", short for number only used once) generated at that point, which is incorporated into the hash, so replay attacks (where they just capture the encrypted packets sent by a client and send them again to login as that user) are much more difficult to achieve. The client then takes the password from the user and hashes it with the Argon2id (<https://en.wikipedia.org/wiki/Argon2>) hash function twice. The first time uses the salt as normal, the second time uses the nonce the server generated as the salt. This incorporates the nonce into the hash so basic replay attacks would fail. The server then takes the password hash out of the database (it is stored as the password hashed with that account's salt) and hashes that with the nonce it generated. Finally, it compares the hash it made and the hash sent by the client and if they match, the client logs in and is added to the queue. If they do not match, the server denies access.

---

## 3.2.6 The Database

---

In order to store the user data, I used a SQLite3 database. This is because it doesn't need a database server and there is an existing Python module for it. I knew I needed to secure this data, so I encrypted it. Unfortunately, the Python module I used - `sqlite3` - didn't support full-DB encryption, so I encrypted each field before it went into the table. I used the AES-GCM encryption algorithm to do this, because it is near impossible to decrypt without the key. The password to encrypt and decrypt the data isn't stored anywhere, as the server asks the admin for the password when it first starts. This means even if someone has access to the database and can read all of the code and files, they still won't be able to decrypt the database, however, if the admin forgets the password then there is no way to get the data back. I decided this was a reasonable tradeoff.

---

## 3.2.7 The Game

---

Once someone logs in, their connection is added to a queue. Another thread is watching this queue, and when there are 2 or more connections in the queue, the first 2 are put in a game. The server tells each client the other's name, then runs the first game round. The client has to tell the server when the user has rolled the dice, at which point the server tells both clients what was rolled and what the scores are. This repeats for 5 rounds, then the server checks for a tie and does the tiebreaker if needed. Finally, the server gives each client the final scores and the high scores - the top 5 players and how many points they have got out of all of the games they've played. Each player's information is updated in the database, which holds their total score, how many games they've won and how many games they've lost. When waiting for the users to roll the dice, the server has a 2 minute timeout so if one player just doesn't do anything, the game will still continue.

---

## 3.2.8 The Client

---

### *CLI*

The client was quite simple to start with, as it had a simple command line interface. It just printed the scores and used Python `input()` to wait for the user to press Enter to roll the dice. However, I had some extra time at the end so I upgraded it to a TUI (Textual User Interface).

## TUI

I used the Textual (<https://textual.textualize.io/>) library to achieve this. When the game starts, it prompts you to select a host, with a few options in a radio button set and an Other option where you enter the hostname manually. It then connects, and you enter your username and password and press either Login or Signup. If you press Login, it communicates with the server and logs in, otherwise it asks you for your name and then signs you up and logs you in. There is then a waiting screen while you are in the queue, then it drops you into the main game interface. It shows you your name, your opponent's name, your roll, their roll, the points history of the game, the reason for getting points from a roll, and the roll button. When either player rolls dice, the numbers in the dice squares change rapidly for a few seconds, then slow down until they stop on your roll. When this happens, it shows you how you gained points, with each reason appearing after a second of delay to build tension, then your score is added to the history. The tiebreaker has the same interface, then when showing results it uses a scrolling text effect to build more tension, then the high scores appear one by one.

---

## 3.2.9 Challenges

- Running multiple games simultaneously
- Implementing secure authentication
- Completing the project within the time limit
- Designing the client-server protocol
- Keeping the server in control of everything so clients can't cheat

---

### 3.2.10 What I Learnt

- How real-world apps are kept secure
- How to use basic SQL commands through Python
- Building TUIs with Textual
- Why hashing, symmetric encryption and asymmetric encryption are used
- How to use threading in Python

---

### 3.2.11 What I'd Improve

- I would improve the error reporting in the UI, because it currently just gives the user the Python exception if it fails to connect, or the slightly more friendly server error if it fails to login
- I would try to move to asynchronous networking instead of threading, because it could perform more efficiently when multiple connections are waiting for input
- I would add a mechanism for the server to give its certificate to the client securely (probably needing to use a certificate authority) because currently to connect to your own server you have to manually install the certificate
- I would do more testing as I only tested it manually a few times because I didn't have enough time to write unit or full-stack tests

## 3.3 SamSpeak

2025 - Aged 14

```

1  import os
2
3
4  class Preprocessor:
5      def __init__(self, filename, SamSpeak, code):
6          self.filesRead = [filename]
7          self.SamSpeak = SamSpeak
8          self.code = code.split('\n')
9
10     def preprocess(self):
11         self.code = '\n'.join(self.code)
12         self.code = self.code.replace("\n", '\n')
13         self.code = self.code.split('\n')
14         for num, line in enumerate(self.code):
15             if line.startswith("#"):
16                 self.code[num] = ""
17                 if not os.path.exists(f"{line[1:]}"):
18                     self.SamSpeak.scanError(num+1, "This file doesn't exist!")
19                     continue
20                 if f"{line[1:]}." in self.filesRead:
21                     continue
22                 else:
23                     self.filesRead.append(f"{line[1:]}")
24                 with open(f"{line[1:]}") as file:
25                     lineNumber = 0
26                     for line in file.readlines():
27                         self.code.insert(num+lineNumber, line.strip())
28                         lineNumber += 1
29                 self.code = self.preprocess()
30             elif line.startswith("!"):
31                 self.code[num] = ""
32                 self.SamSpeak.interpreter.addModule(line[1:])
33                 self.code = self.preprocess()
34         return self.code
35

```

```

Factorial iterative: 362880
Factorial recursive: 362880
(venv) [arcco@alarm] ~/Documents/Python/SamSpeak
$ python3 SamSpeak.py factorial.ss
Enter a number: 12
Factorial iterative: 479001600
Factorial recursive: 479001600
(venv) [arcco@alarm] ~/Documents/Python/SamSpeak
$

```

A sample of part of the code

### 3.3.1 Overview

SamSpeak is a programming language and interpreter written in Python. I implemented a lexer, recursive-descent parser, AST, resolver and interpreter, then extended the language with arrays, maps, modules and exception handling. I found and followed a tutorial at Crafting Interpreters (<https://craftinginterpreters.com/contents.html>). I mostly followed the tutorial, but I made it in Python instead of Java as I was more comfortable with it. I then changed a large chunk of the syntax and added more features (such as structures, modules, arrays and maps) so it was *my* language, and I learnt how the sections of the program worked together. The code is on my GitHub (<https://github.com/SamBell2/SamSpeak>).

### 3.3.2 Preprocessor

This was one of the parts that I added that wasn't in the tutorial, and as such it isn't as good. It does three things:

- **Escape characters:** It replaces text such as `\n` with an actual newline.
- **User modules:** Any line starting with `#` is replaced with the contents of the file at the path after the `#`, ignoring if already included or a circular import.
- **Builtin modules:** Any line starting with `!` is removed and the interpreter includes any functions in the corresponding builtin module.

---

## 3.3.3 Lexer

This is the part of the program that takes the raw characters and groups them into tokens. For example, if it finds the string `i = k + 1` it will create this list:

- IDENTIFIER `i`
- EQUALS
- IDENTIFIER `k`
- PLUS
- NUMBER `1`

It is relatively simple, mostly just a loop with a big `if...elif...else` block inside.

---

## 3.3.4 Parser

This is the first complicated section, because it turns an array of tokens into an AST - Abstract Syntax Tree - that describes the program flow, conditionals, variables and more. It represents what the entire program is meant to do in a simple structure. I used a recursive descent parser, which is a collection of functions that call each other in order. The first one is `declaration` which looks for a variable, structure or function declaration. If it finds one, it produces the corresponding node in the tree to represent declaring that item. If it doesn't find one, it calls `statement` which does a similar thing but for conditionals, loops, etc. This keeps going down the chain until a function finds what it needs or it reaches the end. If the token reaches the bottom function (`primary` - numbers, string literals, booleans, groupings or a few select keywords) and hasn't been consumed, there is a syntax error. This approach is relatively simple as it handles precedence easily. The higher-precedence expressions are simply at the end of the chain, so can't consume a lower-precedence expression by mistake.

---

## 3.3.5 Resolver

This part was weird. It determined which declaration each variable refers to, became important when variables have the same name in different scopes. Most programming languages support something called *closures*, which go something like this:

```
var a = "global";
{
  fun showA() {
    print a;
  }

  showA();
  var a = "block";
  showA();
}
```

This is supposed to print "global" twice, as it does in most languages (not Python because in Python the second declaration of `a` reassigns it instead of declaring it), but instead it printed "global" then "block". This needed a lot of work to fix. Essentially I used the same structure as an interpreter, but didn't do anything for most nodes. Nodes that accessed some sort

of variable, however, needed more effort. I sort of 'marked' on each variable which declaration it was meant to point to, so in the `print a;` line above it would point to the global declaration so printed "global" twice. It felt like a lot of effort to change something virtually no-one would use anyway, but I still did it to learn how.

---

## 3.3.6 Interpreter

---

This part wasn't as bad as I was expecting. It recursively evaluates the AST. Each type of node has its own behaviour: arithmetic expressions calculate values, variable nodes access stored values, and statement nodes perform actions such as loops, conditionals and function calls. It essentially just executes the first node in the AST, then if that node had any subnodes, it would execute the first one, then that one's first subnode, then second, and so on until the entire program had been executed.

---

## 3.3.7 My Changes and Additions

---

### *Syntax*

In the tutorial language, statements ended in semicolons, which I just removed. I have no doubt that some bugs now exist with this but I haven't been able to find any. I also changed the `{ }` blocks to `;` just because I wanted to.

### *Language Features*

I removed classes and replaced them with structures which I think are more elegant. I added `try...catch` blocks and `raise` statements. I added a few more operators including modulus and powers.

### *Other Features*

I added modules so you can split large programs into smaller, simpler blocks. I also changed the error system so it insults you when there is an error, because I was bored a few months later.

---

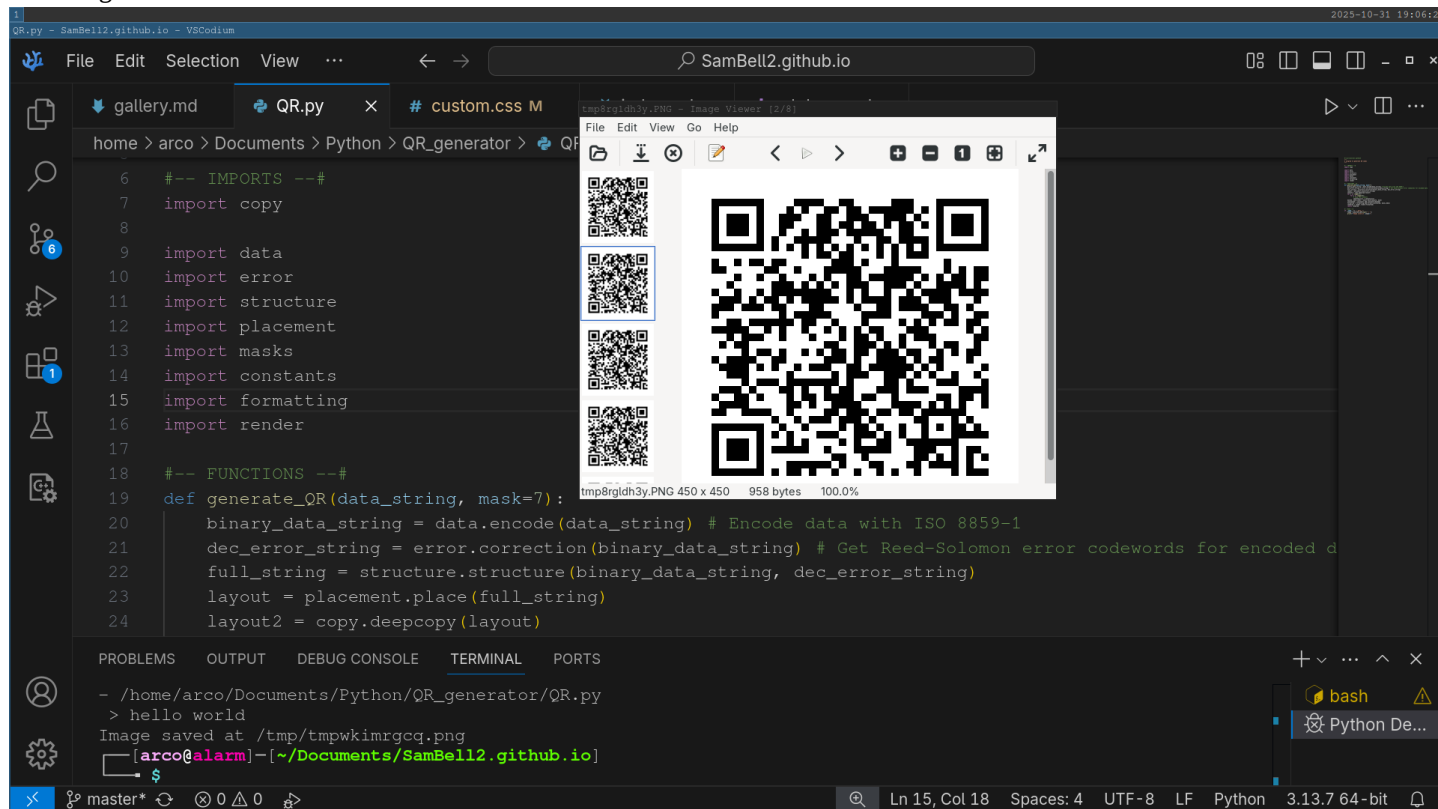
## 3.3.8 Challenges

---

- Fixing closures - for a while I didn't even know this was a bug
- Converting the Java structure into Python - two different categories of language
- Tracking line numbers after adding modules - inserting user modules messed up error reporting

## 3.4 QR Code Generator

2024 - Aged 13



Part of the code and the output of the QR code generator

### 3.4.1 Overview

I got the inspiration for this project because QR codes are everywhere but very few people actually think about how they work. I then found an amazing guide (<https://www.thonky.com/qr-code-tutorial/>) that showed every step. QR codes are far more complicated than you would think, mostly due to the error-correction. You can imagine that if you scan a QR code in not ideal conditions, then the camera may misread one or two pixels. This means we need error correction, which is really complex! The code is on my Github ([https://github.com/SamBell2/QR\\_generator](https://github.com/SamBell2/QR_generator)).

### 3.4.2 What It Does

It takes a line of text as input from the user, then generates the QR code, saves it as a temporary file and opens it in your default image viewer. From there you can save it somewhere else or share it.

---

## 3.4.3 Technical Summary

It is made of almost 700 lines of Python code split over 9 files. I have used no external modules relating to QR codes, only `pillow` for graphics, `copy` to duplicate it and `tempfile` to save the image. It generates a v5 error level Q alphanumeric QR code, which can hold 87 characters.

---

## 3.4.4 Steps

There are multiple steps in making a QR code:

1. Encode the text in a particular text encoding (ISO 8859-1)
  2. Generate error correction codewords
  3. Interleave the data and error codewords to make a final binary string
  4. Place the individual bits of the binary string in the correct places
  5. Apply the mask that makes the QR code easier to read
  6. Add constant elements (e.g. the corners)
  7. Render
- 

## 3.4.5 Encoding

This is one of the simplest steps, as all you have to do is connect the mode string (0100 for text) to the length of the string in binary. You then have to convert each character in the text to the ISO 8859-1 standard. This is easier than I thought it was going to be as it is the standard that Python's `chr()` function uses. After that, you just need to fill out the rest of the space with a repeating pattern.

---

## 3.4.6 Error Correction

This is by far the hardest part of the QR code. It has quite a lot of maths involved, including polynomial long division in a Galois Field 256. You have to convert the data string to a message polynomial and get a generator polynomial. You then have to divide the message polynomial by the generator polynomial, and the coefficients of the remainder polynomial are the error correction codewords.

---

## 3.4.7 Structuring

You have to break the data codewords into blocks, then interleave the first codeword from the first block, followed by the first from the second block and so on. Once you have done the first codeword from each block, you then move onto the second from each block. Once you have done all the data, you then do the same with the error correction codewords. You then need to add a certain number of zeroes to the end so the QR code will be full.

---

## 3.4.8 Placement

---

This is a relatively simple part, although still more complex than you would imagine as there are a number of pixels that you have to avoid. You just start at the bottom-right of the QR code and work your way up in a zig-zag pattern, avoiding all of the reserved areas.

---

## 3.4.9 Masks

---

Occasionally, when you make a QR code, there will be areas of data that look like special symbols such as the corner patterns. This would really confuse the scanner, so you have to apply a mask. These are just lists of which pixels to invert. You have to apply all of them and evaluate which is the best (i.e. has the fewest confusing patterns) then apply that one.

---

## 3.4.10 Constants

---

This is the simplest bit as most of it is pre-set. You have to add the corners, borders, alignment patterns, timing patterns, dark module and the format information areas. The format information needs to be generated but isn't hard. The format string is mostly the same for my codes as I have only one type of QR code. The only bit that changes is the mask pattern used. You then have to error-correct it and add it.

---

## 3.4.11 Render

---

For this, I use Python's `pillow` library to work with images. Each pixel of the QR code is 10 pixels wide in the image so when I open it in an image viewer, it doesn't blur the edges between pixels. I just loop over each pixel in the QR code and place a 10x10 square of whatever colour it is in the image.

---

## 3.4.12 Challenges

---

The hardest bit of this was the error correction, as at the time I didn't even know what a polynomial was or how binary worked. I had to write code to do maths with strings in weird number systems. It took me about a week to make 160 lines of code, and another week to test and debug it!

## 3.5 Scripted Journeys

2024-2025 - Aged 13-14

```

arc@kali:~$
You see an exit to the North.
You see an exit to the South.
You see an exit to the East.
> move north

Crystal Grotto
Crystals embedded in the walls glow with a soft, ethereal light, creating a mesmerizing display.
There are no items here.
A massive golem made of living crystal, slow but immensely strong.
You see an exit to the North.
You see an exit to the South.
You see an exit to the East.
> fight golem

A battle begins with the Golem!
You hit the Golem with your Fists. It causes 0.5 damage.
The Golem hits you with its Runic Hammer. It causes 3 damage.
HP: [██████████] 7/10 HP
Continue?
> yes
You hit the Golem with your Fists. It causes 0.5 damage.
The Golem hits you with its Runic Hammer. It causes 3 damage.
HP: [██████████] 4/10 HP
Continue?
> yes
You hit the Golem with your Fists. It causes 0.5 damage.
The Golem hits you with its Runic Hammer. It causes 3 damage.
HP: [██████████] 1/10 HP
Continue?
> 

```

*A section of the game in the middle of a fight*

### 3.5.1 Overview

Scripted Journeys is a text-based adventure game. I originally got the idea from a programming article which made a similar one, which I then improved on. I added multiple maps, which are defined as a set of XML files so users could add new ones if they want. I also added user settings and a few new features. Finally, the hardest part about the whole project was packaging and publishing it so people can actually play it.

#### Links

- **GitHub:** <https://github.com/SamBell2/ScriptedJourneys> (<https://github.com/SamBell2/ScriptedJourneys>)
- **Flathub:** <https://flathub.org/en/apps/io.github.MrPiggy92.ScriptedJourneys> (<https://flathub.org/en/apps/io.github.MrPiggy92.ScriptedJourneys>)
- **Windows Installer:** Download (<https://github.com/SamBell2/ScriptedJourneys/raw/refs/heads/master/windows/Output/Scripted-20Journeys.exe>)

## 3.5.2 How It Works

---

The first thing it does when you open the game is it checks if you have already played it. If this is the first time you play it, it will give you an introduction and the option to change some settings. It then opens the main screen. Once you have chosen a map from the main screen, it launches the actual game.

---

## 3.5.3 Main Screen

---

This is a simple loop, getting input then using a large if-elif-else block to process it. It supports a few different functions:

- **Listing available maps:** It just uses `os.listdir()` with the map folder and filters the output.
  - **Changing settings:** It gets user settings from a JSON file, then lets the user select which setting to change and what to change it to, before writing it back to the file.
  - **Loading a saved game:** It has a single save file, which it loads and decodes using `pickle`. It is a Python list of the Map object then the Player object. Each of these contain references to the other required objects. Once these are decoded, it starts playing using this current state.
  - **Quitting:** It simply raises a `SystemExit()` to close the game. It is terminal-based and all files are closed as soon as they have been written to or read from, so it doesn't need to worry about closing anything special.
  - **Launching the game:** The main screen doesn't need to do anything apart from run a single function. This runs the actual game, and once it is finished the function returns.
- 

## 3.5.4 Game Loop

---

This is similar to the main screen in that it is just a big loop, but it has a few extra tricks. This is what it does every loop:

1. It checks if you are dead.
  2. It outputs a block of information, including what room you are in, where the exits are, the items in the room and what enemies you can fight.
  3. It gets your command.
  4. It parses and runs your command:
    1. It uses a large dictionary that relates the command with the function that runs it and the number of arguments the function takes.
    2. It uses fuzzy matching to find which command you meant to type. This means it doesn't matter if you have a typo. It uses a Levenshtein Distance algorithm to work out how many alterations are needed to convert your command into a possible command, and uses the result.
    3. It retrieves the correct details about your command from the dictionary.
    4. It runs the function with the map and the player as arguments, as well as the second word of the command if the function wants it.
  5. If there is a `RuntimeError` (which I am using as a quit signal) then it breaks out of the loop. I should probably create a separate exception instead of `RuntimeError` but so far it hasn't been necessary.
-

## 3.5.5 Publishing

---

Once I had got the game to a playable state, I decided to publish it online. First, I put it on Flathub (<https://flathub.org>) because I use Linux on my computer, and Flathub is one of the main app stores. Once I had done that, I kept developing it for a while, before I decided to make it work on Windows. I still want to make it work on macOS, but in order to publish apps for Apple, you have to have a Mac and I don't.

### *Flathub*

This was the most challenging part of the project. First, I had to make the app set itself up when you first run it, then I had to make a manifest file. This defined all of the dependencies my app needed, the metadata, where the code was and how to put all the bits of the app in the right places. I then had to test it a lot because I got half of it wrong the first time. Next, I had to submit a pull request to a GitHub repository which added my manifest and code to a new branch. It got rejected a few times, but I eventually discovered that it was because I had to have the code online, not in the PR. It took me about a week to fix the sources section of the manifest to work with a GitHub source, but once I did it got accepted quickly. It is now online here (<https://flathub.org/apps/io.github.MrPiggy92.ScriptedJourneys>).

### *Windows*

The hardest part about packaging for Windows was the refactoring of a good portion of it to remove Linux-only sections, which took a good few weeks. I then had to check that I hadn't broken the Flathub elements, then I spent a while fixing those bits I did break. Once all of the code worked on both of them, I had to compile it to an executable, which contained the actual game. However, I still wasn't finished. This ran the game, but it didn't install it, so I still had to work out how to actually package it. This bit was much easier than I thought, as I found a tool called Inno Setup (<https://jrsoftware.org/isinfo.php>) which did most of the work for me. I still had to configure it so it put all of the files in the right place in the EXE, but it did the hardest bits. You can grab the installer here (<https://github.com/SamBell2/ScriptedJourneys/raw/refs/heads/master/windows/Output/Scripted-20Journeys.exe>).

---

## 3.5.6 Challenges

The only real challenge in the programming part was working out how to get the game files over multiple operating systems, but it wasn't too hard. The hardest part was publishing it to Flathub, because I had to make the manifest and then go through the long and complicated process of getting my app accepted by the maintainers.

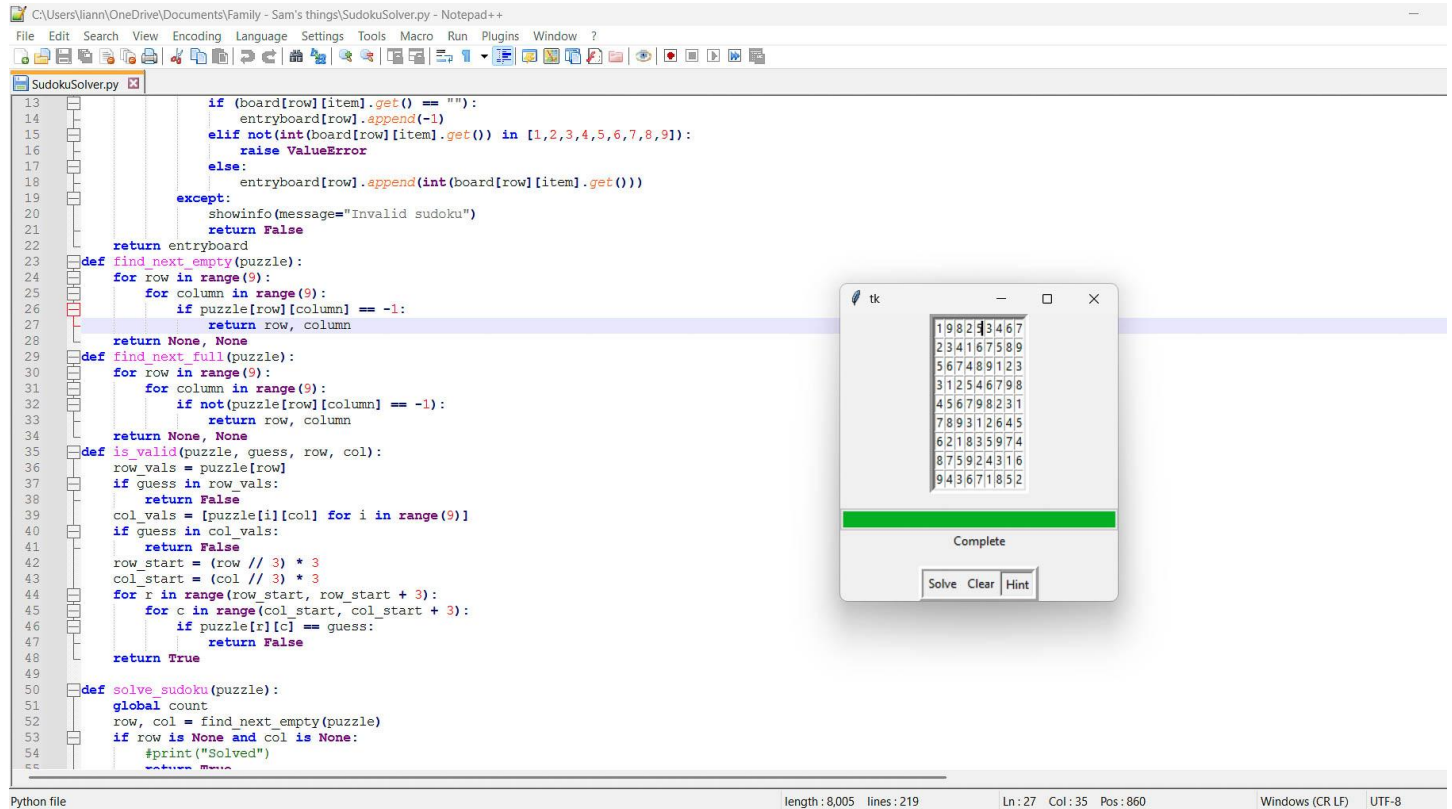
---

## 3.5.7 What I Learnt

- ANSI color codes in terminal output
- Data and settings storage for apps
- Python compilation and packaging
- Cross-platform distribution (Windows & Linux)
- Creating Windows installers
- Publishing apps on Flathub

# 3.6 Sudoku Solver

2023 - Aged 12



A completed sudoku

## 3.6.1 Overview

This was my first project related to computer intelligence: it uses a recursive backtracking algorithm to test every possible solution, rejecting wrong answers as early as possible. You can give it a partial 9x9 sudoku puzzle and it will solve it for you. I learnt a lot about how recursion can be used to solve search-based problems.

## 3.6.2 Features

- Solves 9x9 sudoku puzzles
- Checks for impossible sudoku
- GUI for easy use
- Gives hints to the next square

## 3.6.3 Backtracking Algorithm

---

The solver uses a recursive backtracking algorithm to find the solution. The program begins at the first empty square, setting it to 1. It then checks if the puzzle is impossible (i.e. the same number is found twice in a single row, column or 3x3 box). If the puzzle is possible, the solver then calls itself to solve this new puzzle. If, however, the new puzzle is not possible, it changes the 1 to a 2 and checks again. This repeats until either it finds a possible number or gets to 9. If it gets to a 9, then some previous square is wrong, so it blanks the square out and returns to the solver above it. The solver above it now knows that the numbers it has are not possible, so it adds 1 to the last number and tries again. This whole recursive process keeps going until either the whole puzzle is complete, or it has tried every combination and none of them work. If this is the case, it reports an unsolvable sudoku.

---

## 3.6.4 The Possibility Checker

---

This loops over each square of the puzzle, where it checks if it is empty. If not, if it is in the set of numbers in this row, then it says it is impossible (the same number cannot occur twice in the same row). If it is not in the set, then it is added to the set. It then does the same with columns, and then blocks.

---

## 3.6.5 The GUI

---

I used Tkinter to make the GUI as it is quite simple. First, I made a `Frame` containing 81 `Entry` s in a grid layout. The variables holding these `Entries` were held in a 2D array mirroring the puzzle. I also added a `Progressbar` and 3 `Button` s in another `Frame` . When you press the Clear button, it empties the entire sudoku. When you press the Hint button, it solves the entire puzzle and tells you the next number. When you press Solve, it solves the puzzle, keeping track of the number of iterations and recursions done, then runs the progress bar for a few seconds (it is relative to the number of steps done) before telling you the result. I didn't need the progress bar, and the entire puzzle is done before the progress bar even starts, but I added it to learn more about GUI elements.

---

## 3.6.6 The Code

---

You can see the full implementation below, including the solver, validity checking and the GUI.

▼ Show/Hide Code

```

#!/usr/bin/env python3

## IMPORTS ##

import tkinter as tk
from tkinter import ttk
from tkinter.messagebox import showinfo

## VARIABLES ##

count = 0

## METHODS ##

def board_to_list(board):
    entryboard = [[] for _ in range(9)]
    for row in range(9):
        for item in range(9):
            try:
                if (board[row][item].get() == ""):
                    entryboard[row].append(-1)
                elif not(int(board[row][item].get()) in range(1,10)):
                    raise ValueError
            else:
                entryboard[row].append(int(board[row][item].get()))
        except:
            showinfo(message="Invalid sudoku")
            return False
    return entryboard

def find_next_empty(puzzle):
    for row in range(9):
        for column in range(9):
            if puzzle[row][column] == -1:
                return row, column
    return None, None

def is_valid(puzzle, guess, row, col):
    row_vals = puzzle[row]
    if guess in row_vals:
        return False
    col_vals = [puzzle[i][col] for i in range(9)]
    if guess in col_vals:
        return False
    row_start = (row // 3) * 3
    col_start = (col // 3) * 3
    for r in range(row_start, row_start + 3):
        for c in range(col_start, col_start + 3):
            if puzzle[r][c] == guess:
                return False
    return True

def solve_sudoku(puzzle):
    global count
    row, col = find_next_empty(puzzle)
    if row is None and col is None:
        return True
    for guess in range(1,10):
        count += 1
        if is_valid(puzzle, guess, row, col):
            puzzle[row][col] = guess
            if solve_sudoku(puzzle):
                return True
            puzzle[row][col] = -1
    return False

```

```

def is_possible(puzzle):
    for i in range(9):
        row = {}
        column = {}
        block = {}
        row_cube = 3 * (i//3)
        column_cube = 3 * (i%3)
        for j in range(9):
            if puzzle[i][j] != -1 and puzzle[i][j] in row:
                return False
            row[puzzle[i][j]] = 1
            print(row)
            if puzzle[j][i] != -1 and puzzle[j][i] in column:
                return False
            column[puzzle[j][i]] = 1
            rc = row_cube + j // 3
            cc = column_cube + j % 3
            if puzzle[rc][cc] in block and puzzle[rc][cc] != -1:
                return False
            block[puzzle[rc][cc]] = 1
    return True

def handle_solve_click(event, board, pb):
    global count
    count = 0
    entryboard = board_to_list(board)
    if not entryboard:
        return
    if not(is_possible(entryboard)):
        showinfo(message="Invalid sudoku")
        return False
    solve_sudoku(entryboard)
    time = count / 5
    while time > 10000:
        time -= 1000
        print(time)
    pb.start(round(time/100))
    window.after(round(time), lambda: show_solution(entryboard, board, pb))
    window.after(10, update_progress_bar)

def show_solution(entryboard, board, pb):
    global count
    count = 0
    for row in range(9):
        for item in range(9):
            board[row][item].delete(0, tk.END)
            board[row][item].insert(0, entryboard[row][item])
    print("+ " + "---"*9)
    for i, row in enumerate(entryboard):
        print(("|" + " {}  {}  {} |"*3).format(*[x if x != -1 else " " for x in row]))
        if i % 3 == 2:
            print("+ " + "---"*9)
        else:
            print("+ " + "   "+"*9)
    pb.stop()
    pb['value'] = 100

def handle_clear_click(event, board, pb, progress):
    for row in range(9):
        for item in range(9):
            board[row][item].delete(0, tk.END)
    pb['value'] = 0
    progress['text'] = "0.0%"

def handle_hint_click(event, board):
    entryboard = board_to_list(board)
    otherboard = board_to_list(board)

```

```

if not(entryboard):
    return False
if not(is_possible(entryboard)):
    showinfo(message="possible")
    return False
solve_sudoku(entryboard)
for row in range(9):
    for item in range(9):
        if otherboard[row][item] != entryboard[row][item]:
            board[row][item].delete(0, tk.END)
            board[row][item].insert(0, entryboard[row][item])
            return True
showinfo(message="Already solved")

def update_progress_bar():
    if pb['value'] < 100:
        progress['text'] = f"{pb['value']}%"
        window.after(10, update_progress_bar)
    else:
        pb['value'] = 100
        progress['text'] = "Complete"

## MAIN LOOP ##

if __name__ == "__main__":
    window = tk.Tk()
    board = [[] for _ in range(9)]
    sudoku_frame = tk.Frame(relief=tk.SUNKEN, borderwidth=5)
    for row in range(9):
        for item in range(9):
            myentry = tk.Entry(master=sudoku_frame, width=2, justify='center')
            myentry.grid(row=row, column=item)
            board[row].append(myentry)
    sudoku_frame.pack()
    progress_frame = tk.Frame()
    pb = ttk.Progressbar(master=progress_frame, orient='horizontal', mode='determinate', length=280)
    pb.grid(row=0, column=0)
    progress = tk.Label(master=progress_frame, text="0.0%")
    progress.grid(row=1, column=0)
    progress_frame.pack(pady=15)
    button_frame = tk.Frame(relief=tk.RIDGE, borderwidth=5)
    solve_btn = tk.Button(master=button_frame, text="Solve", relief=tk.FLAT, borderwidth=2)
    solve_btn.bind("<Button-1>", lambda event: handle_solve_click(event, board, pb))
    solve_btn.grid(row=0, column=0)
    clear_btn = tk.Button(master=button_frame, text="Clear", relief=tk.FLAT, borderwidth=2)
    clear_btn.bind("<Button-1>", lambda event: handle_clear_click(event, board, pb, progress))
    clear_btn.grid(row=0, column=1)
    hint_btn = tk.Button(master=button_frame, text="Hint", relief=tk.FLAT, borderwidth=2)
    hint_btn.bind("<Button-1>", lambda event: handle_hint_click(event, board))
    hint_btn.grid(row=0, column=3)
    button_frame.pack()
    window.mainloop()

```

## 3.6.7 Challenges

The most challenging part of the whole process was actually checking if a solution was impossible. This was mostly because the 3x3 blocks were really hard to check through neatly, without hardcoding a check for each square. If I come back to this project in the future, I will probably remove the progress bar and instead update the grid as it is solved, showing the user exactly how it works. However, this would mean either making the solver directly access frontend code, or restructuring it to yield intermediate states.

---

## 3.6.8 What I Learned

---

- How recursive backtracking works to explore every option
- How to reject invalid solutions early
- How to separate backend logic from frontend displays
- How to make simple GUIs

## 3.7 Morse Code Translator

2021 - Aged 10

```
morse.py - SamBell2.github.io - VSCodeium
2023-10-31 19:08:05

File Edit Selection View ... SamBell2.github.io
gallery.md morse.py x QR.py # custom.css M index.md mkdocs.yml
home > arco > Documents > Python > morse > morse.py > ...
1 class Translator():
12     def fromMorse(self, msg):
19         for letter in letters_in_word:
20             try:
21                 message_in_english += letters[letter]
22             except:
23                 message_in_english += '#'
24         message_in_english += ' '
25         return message_in_english
26
27 if __name__ == "__main__":
28     translator = Translator()
29     msg = input("Enter message to translate to morse: ")
30     print(translator.toMorse(msg))
31     msg = input("Enter message in morse to translate to English: ")
32     print(translator.fromMorse(msg))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[X] — [arco@alarm] — [~/Documents/SamBell2.github.io]
tensions/ms-python.debugpy-2025.14.1-linux-arm64/bundled/libs/debugpy/adapters/.../debugpy/launcher 57435 -- /
home/arco/Documents/Python/morse/morse.py
Enter message to translate to morse: hello world
.... . -.-. .-.-. --- ..-
Enter message in morse to translate to English: .... . -.-. .-.-. --- ..-
hello world
[arco@alarm] — [~/Documents/SamBell2.github.io]
$
```

A section of the code and its output

### 3.7.1 Overview

This was one of my first projects, it started just with taking input in English and translating it to morse code. I then expanded it by reversing it, so it could take input as morse code and return English. Next, I learnt the basics of OOP (Object Oriented Programming) and turned it into an object with 2 methods. Looking back, there was no real reason for this other than to experiment with how classes worked. I then used a Raspberry Pi and an LED so I could input English and the light would flash the morse code. Finally, I tried (but failed) to reverse the electronics, so I could point a camera at an LED and it would tell me what it was saying.

### 3.7.2 The Translator

I defined an enormous Python dictionary relating letters, numbers and symbols to their corresponding morse codes:

```
{'a': '-.-.', 'b': '-...-', 'c': '-.-.-.', 'd': '-.-.', 'e': '.', 'f': '..-.-.', 'g': '--.-', 'h': '....', 'i': '...', 'j': '.---', 'k': '-.-.-', 'l': '-.-.-.', 'm': '---', 'n': '-.-', 'o': '---', 'p': '..-.-.', 'q': '-.-.-.-', 'r': '..-.-', 's': '...', 't': '-.-', 'u': '..-.-', 'v': '..-.-.-', 'w': '-.-.-.-', 'x': '-.-.-.-.', 'y': '-.-.-.-.-', 'z': '-.-.-.-.-.', '1': '-.-.-.-.-', '2': '-.-.-.-.-.-', '3': '-.-.-.-.-.-.-', '4': '-.-.-.-.-.-.-.-', '5': '-.-.-.-.-.-.-.-.-', '6': '-.-.-.-.-.-.-.-.-.-', '7': '-.-.-.-.-.-.-.-.-.-.-', '8': '-.-.-.-.-.-.-.-.-.-.-.-', '9': '-.-.-.-.-.-.-.-.-.-.-.-.-', '0': '-.-.-.-.-.-.-.-.-.-.-.-.-.-', '?': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '!': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', ':': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', ';': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', ',': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '.': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '/': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '_': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '=': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '+': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '-': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '_': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', '~': '-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-', ' ': ' '}
```

I then looped through the characters in the string to translate, converted them to lowercase, and retrieved the code. If the character wasn't in the map, I used a # symbol. I then added the code to a string, followed by a space. Finally, I returned the string. To convert it back into English, I did something similar. I used a reversed version of the dictionary, so it mapped the morse codes to the letter, then I split the morse message into words. I then split each word into characters, retrieved the letter for each morse character, and added a space between each word.

## 3.7.3 The Object

This was my first attempt at object-orientation, and I didn't use it properly. I just used a class to hold two methods - `toMorse` and `fromMorse` - and that was it. I made a `Translator` class, which I then instantiated, then called the methods on the instance. To be precise, here is the code I had at this stage:

```
class Translator():
    def toMorse(self,msg):
        message_in_morse = ''
        letters = {'a':'.-.', 'b':'-...', 'c':'-.-.', 'd':'-..', 'e':'.', 'f':'.-.-.', 'g':'--.', 'h':'. ....', 'i':'. .', 'j':'.---', 'k':'.-.-',
        'l':'.-..', 'm':'.--', 'n':'.-.', 'o':'---', 'p':'.-.-.', 'q':'--.-.', 'r':'.-.', 's':'. ...', 't':'-.', 'u':'.-.-', 'v':'.-.-.', 'w':'.--', 'x':'.-.-.',
        'y':'.-.-.', 'z':'.--.', '1':'.-----', '2':'.----', '3':'.---', '4':'.--', '5':'.-', '6':'.-.-.-', '7':'-----', '8':'.-----', '9':'.-----',
        '0':'.-----', '?':'.-.-.-.', '!':'.-.-.-.', ':':'.-.-.-.', ',':'.-.-.-.', ';':'.-.-.-.'}
        for letter in msg:
            try:
                message_in_morse += letters[letter.lower()]
            except:
                message_in_morse += '#'
        message_in_morse += ' '
        return message_in_morse
    def fromMorse(self,msg):
        letters = {'.-': 'a', '-...': 'b', '-.-.': 'c', '-..': 'd', '.': 'e', '.-.-.': 'f', '--.': 'g', '....': 'h', '. .': 'i', '.---': 'j', '.-.-': 'k',
        '-.-.': 'l', '.--': 'm', '.-.': 'n', '---': 'o', '.-.-.': 'p', '--.-': 'q', '.-.': 'r', '. ...': 's', '-.': 't', '.-.-': 'u', '.-.-.': 'v', '.--': 'w', '.-.-.': 'x',
        '-.-.': 'y', '.--.': 'z', '.-----': '1', '.----': '2', '.---': '3', '.--': '4', '.-': '5', '.-.-.-': '6', '-----': '7', '.-----': '8', '.-----': '9',
        '.-----': '0', '.-.-.-.': '?', '.-.-.-.': '!', '.-.-.-.': ':', '.-.-.-.': ',', '.-.-.-.': ';' }
        message_in_english = ''
        message_words = msg.split(' ')
        #print(message_words)
        for word in message_words:
            letters_in_word = word.split(' ')
            for letter in letters_in_word:
                try:
                    message_in_english += letters[letter]
                except:
                    message_in_english += '#'
            message_in_english += ' '
        return message_in_english

if __name__ == "__main__":
    translator = Translator()
    msg = input("Enter message to translate to morse: ")
    print(translator.toMorse(msg))
    msg = input("Enter message in morse to translate to English: ")
    print(translator.fromMorse(msg))
```

## 3.7.4 The Electronics

---

Unfortunately, I lost the code I had for the lights, but I think I just attached a simple LED to my Raspberry Pi and blinked it. For a dot, it would light up for 0.1 seconds, then turn off again. For a dash, it was 0.5 seconds. The light was off for 0.3 seconds between each dot and dash, and it was off for 1 second between letters. Between words, I think it stayed dark for 2 seconds. I could certainly have sped this up, but I wanted to make a reader and I thought it would be easier to have slower flashes. I never managed to make the reader, because I wanted to use a camera, not a light dependant resistor, so I could make an app for a mobile phone. I asked StackOverflow how to do this (you can see my question here (<https://stackoverflow.com/questions/68589312/can-you-translate-a-flashing-light-into-morse-code>) - I didn't know how to ask questions well then) and there was an amazing answer (<https://stackoverflow.com/a/68859624/16398595>) involving calculating frame-by-frame average brightness levels, converting it to a binary signal with a threshold value, retrieving the length of each flash and the time between flashes, classifying them with a threshold, and reconstructing the signal. Sadly, I wasn't good enough at programming back then to understand the idea, but it was a great answer.

---

## 3.7.5 Improvements

---

If I was to do this project again, I would fix a few things:

- I would remove the object as it adds unnecessary complexity
- I would add 2 versions: one for a Raspberry Pi and one for a normal computer
- The RPi version would have an LED and an LDR to flash and detect flashes:
- The normal version would have a maximised window that changes colour between black and white to flash, and would use the camera to detect flashes
- I would try to make it into a mobile phone app so people can type a message in and point the screen at a friend, who would zoom in on the screen and would receive the message.

# 4. Electronics

---

## 4.1 CPU

---

### 4.1.1 CPU

2025 - Aged 14



*An image of my CPU so far*

### Overview

This is my current project, it is an 8-bit CPU built out of individual ICs (Integrated Circuits). I got the idea in my Computer Science lessons at school, where I learnt how a CPU works and realised that it isn't actually as complex as I thought. I then started to make parts of it to test in Minecraft, before finding Ben Eater's similar project (<https://www.youtube.com/playlist?list=PLowKtXNTBypGqImE405J2565dvjafglHU>) and using that as a guide to making the basic parts of the computer.

### Specs

- 6 - 48 Hz variable clock speed
- 1 core
- 64kB RAM
- 16 registers; r0 is a zero register

- 8-bit maths
- Von Neumann architecture

## My Aim

I want to make an 8-bit CPU with 16 instructions. It will be fully Turing-complete. It will be able to do maths, store data, jump and branch to other parts of a program, and input and output to other devices. The instruction set I want to implement are adding, subtracting, NORing, right shifting, loading from RAM, storing to RAM, loading from a port, storing to a port, loading immediate values, adding immediate values, jumping around the program, branching to other parts of a program depending on a condition, and halting.

## ISA

I have made an ISA (Instruction Set Architecture) for my CPU. It shows all of the instructions that my CPU will be able to run, how to use them, what they do and what they compile to.

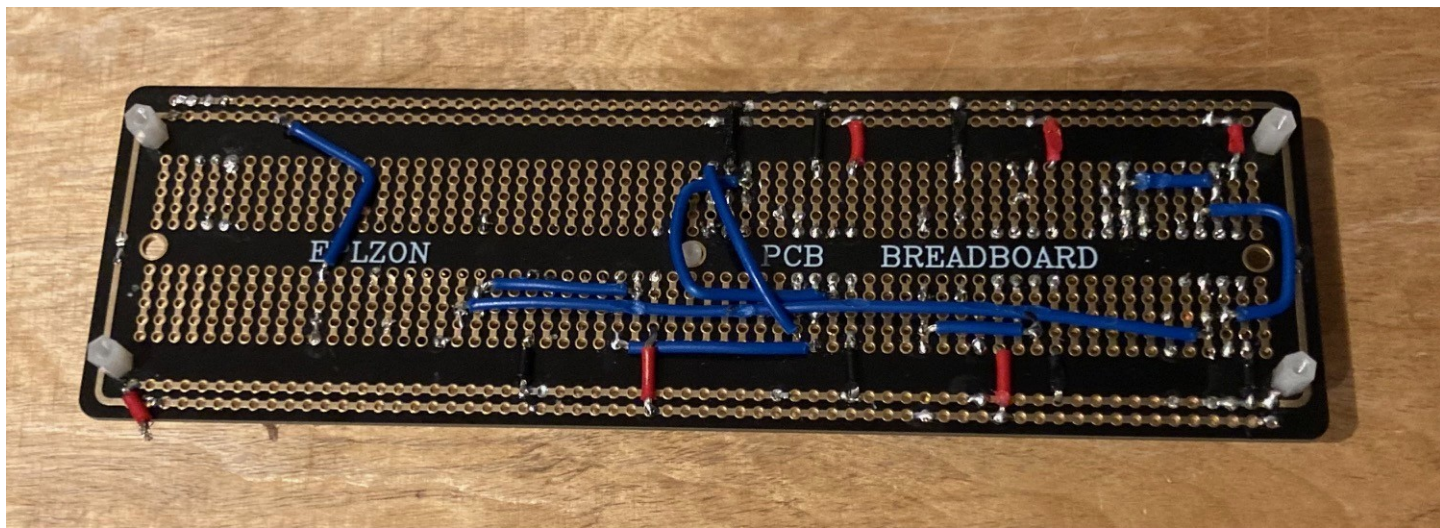
| Opcode | Usage        | Description                 | Binary               |
|--------|--------------|-----------------------------|----------------------|
| ADD    | ADD rX rY rZ | rX + rY -> rZ               | 0000 XXXX YYYY ZZZZ  |
| SUB    | SUB rX rY rZ | rX - rY -> rZ               | 0001 XXXX YYYY ZZZZ  |
| NOR    | NOR rX rY rZ | rX with rY -> rZ            | 0010 XXXX YYYY ZZZZ  |
| RSH    | RSH rX rZ    | right shift rX -> rZ        | 0011 XXXX 0000 ZZZZ  |
| LDR    | LDR rX rZ    | contents of ram at rX -> rZ | 0100 XXXX 0000 ZZZZ  |
| STR    | STR rX rZ    | contents of rZ -> ram at rX | 0101 XXXX ZZZZ 0000  |
| SMB    | SMB rX       | rX -> MBR                   | 0110 XXXX 0000 0000  |
| INP    | INP pX rZ    | pX -> rZ                    | 0111 XXXX 0000 ZZZZ  |
| OUT    | OUT pX rZ    | rZ -> pX                    | 1000 XXXX ZZZZ 0000  |
| JMP    | JMP x        | PC + X -> PC                | 1001 00 XXXXXXXXXXXX |
| BRA    | BRA c X      | if (c) then PC + X -> PC    | 1010 cc XXXXXXXXXXXX |
| ???    |              | undecided                   | 1011                 |
| ???    |              | undecided                   | 1100                 |
| ADI    | ADI rX Y     | add Y to rX                 | 1101 XXXX YYYYYYYYYY |
| LDI    | LDI rX Y     | Y -> rX                     | 1110 XXXX YYYYYYYYYY |
| HLT    | HLT          | halt                        | 1111 0000 0000 0000  |

I also have some pseudo-instructions that the CPU itself cannot execute, but the assembler (the program that turns these commands into binary) will change into commands that can be executed.

| Opcode | Usage     | Description           | Resolves to  |
|--------|-----------|-----------------------|--------------|
| NOP    | NOP       | do nothing            | ADD r0 r0 r0 |
| CMP    | CMP rX rY | compare and set flags | SUB rX rY r0 |
| MOV    | MOV rX rZ | copy rX to rZ         | ADD rX r0 rZ |
| CLR    | CLR rX    | clear rX              | ADD r0 r0 rX |
| INC    | INC rX    | increment rX          | ADI rX 1     |
| DEC    | DEC rX    | decrement rX          | ADI rX -1    |
| NOT    | NOT rX rZ | invert rX -> rZ       | NOR rX rX rZ |
| LSH    | LSH rX rZ | left shift rX -> rZ   | ADD rX rX rZ |

## How I'm Building It

I am making it module by module, first breadboarding it to test it and make sure it works, then soldering it down to a solderable breadboard (<https://www.amazon.co.uk/Prototype-Solderable-Breadboard-Electronics-Gold-Plated-2-Matte-Black/dp/B082KY5Y5Z?th=1>). So far, I have completed the clock module, and mostly done the ALU. I am soldering the components to the top of the board so you can see them easily and the wires to the bottom of the board, just because I think it looks neater.



The back of my CPU clock with all the wires

## What I'm Learning

I am learning LOADS about the internal mechanisms of how computers actually work at the most detailed level, as well as soldering skills, PCB design, and electronic principles such as why pull-up/down resistors are actually necessary.

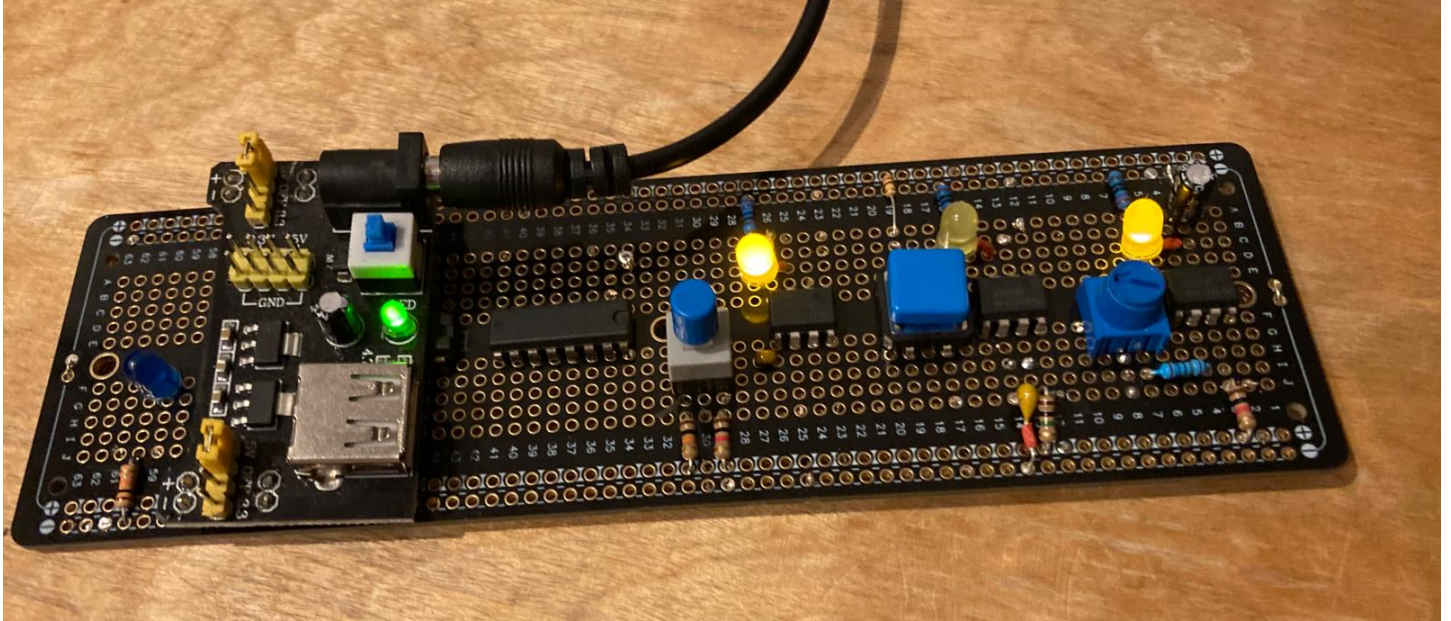
## More Details

- Clock Module

- ALU

## 4.1.2 CPU Clock

2025 - Aged 14



*An image of my CPU clock*

### Overview

In a CPU, it needs something to tell it when to do things. This is the clock. Most computers run at a speed of around *3GHz* (3 *billion* cycles per second). Mine will run at up to 48Hz. The clock is made of 4 parts: the automatic clock, the manual clock, the mode selector and the clock logic. The automatic clock just turns on and off again at a variable speed, set by a potentiometer (a dial). The manual clock pulses whenever you push a button, mainly for debugging and testing. The mode selector lets you pick which mode to use and the clock logic actually does the switching.

### The Automatic Clock

This was the simplest bit, as it was just a 555 timer in astable mode. I used a potentiometer between pins 6 & 7 so I could change the speed. The rest of the connections were quite straightforward.

### The Manual Clock

This was far more difficult than I had anticipated, as buttons often bounce, triggering multiple pulses when you only wanted one. I had to use a 555 timer in monostable mode, so when you push the button, it turns on and 0.3 seconds after you release the button, it turns off. This means that if the button does bounce, the extra pulses will be caught in the 0.3 seconds before it turns off.

## *The Mode Selector*

Both this and the clock logic could be replaced by a simple 3-pin switch, but again you get the problem of button bounce. Instead, I had to use a 555 timer in bistable mode, where you can push a button once and turn it on, and push the button again and turn it off. This was the hardest part, and I took a while to get it working. After a while, I found that you should add a 100nF capacitor between pin 5 & ground, so I did and it worked.

---

## *The Clock Logic*

Now that I have 3 signals (automatic clock, manual clock and mode), I had to make a circuit to switch between auto and manual modes. This wasn't too hard. First, I had to invert the mode so I had a mode signal and a not mode signal. I then ANDed the auto clock signal with the not mode signal, and the manual clock with the mode signal. I then ORed the two outputs together with a homemade diode OR gate. This was my first problem, as it either gave a high output or a floating output, which was registered as high by the next gate. I fixed it by adding some pull-down resistors. Finally, the output of that gets ANDed with the inverted halt signal to get the output.

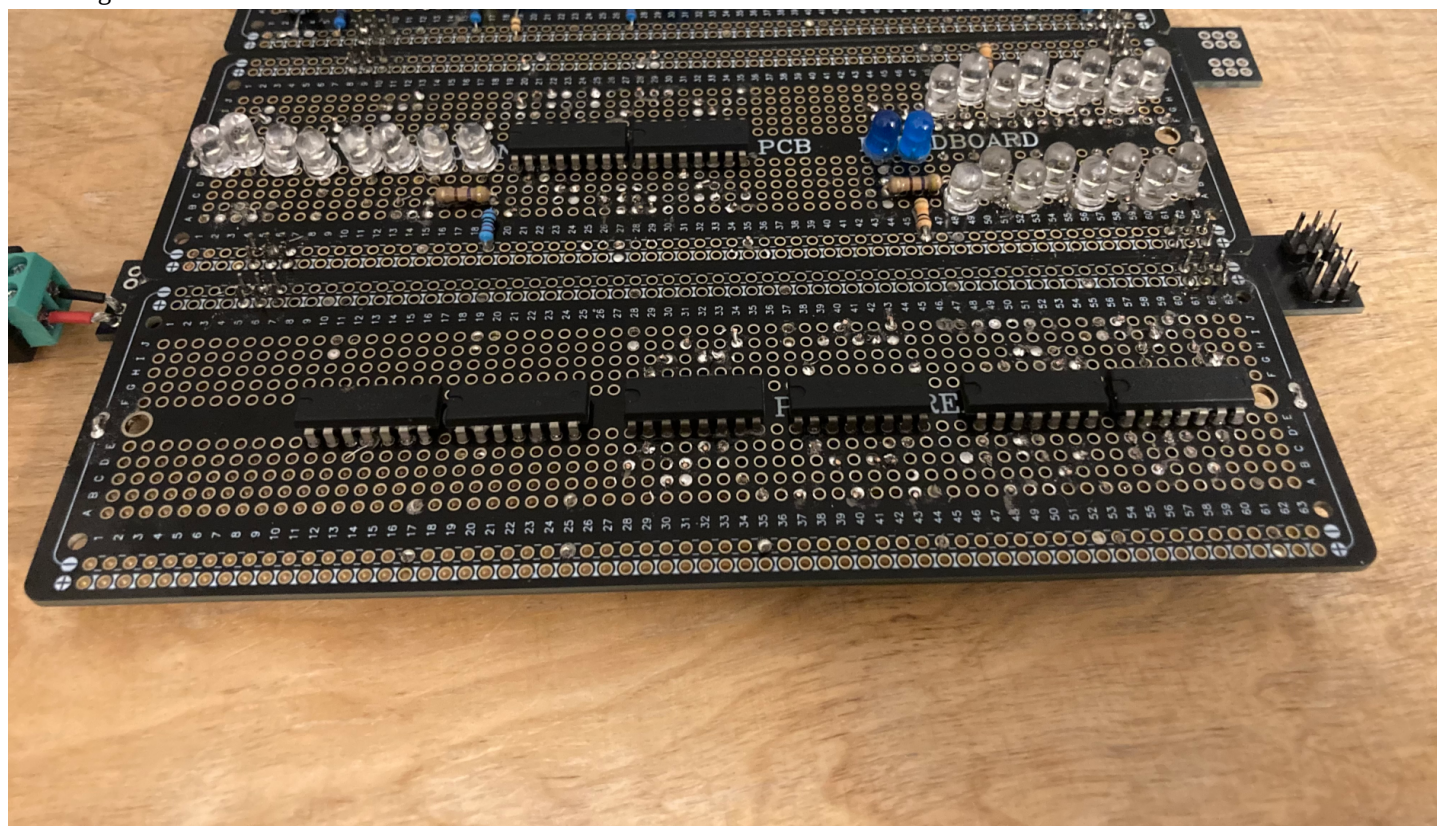
---

## *Challenges*

When testing the clock module, the circuit that should have made the bistable 555 timer didn't work, so I had to experiment with a lot of different methods. I eventually got it working by tying a 0.1 $\mu$ F capacitor across pin 5 and ground, but weirdly when I soldered it down, it worked without the capacitor. I still added it in just in case. In addition, I had a problem with floating outputs of the diode OR gate, but I fixed it with a pull-down resistor. However, sometimes now when it is on the automatic clock, it just stays off. I think I just need to reduce the strength of the resistors, but I haven't done this yet.

## 4.1.3 ALU

2025 - Aged 14



An image of my ALU

### Overview

Doing maths is one of the main purposes of a CPU, and the ALU (Arithmetic and Logic Unit) is the bit that actually does it. As with the rest of my CPU, this is less powerful than most modern ones. It has 4 modes, add, subtract, NOR, and right shift (divide by two, ignore any remainder). I am making it on 2 breadboards: the top one to show the inputs, output, and setting, as well as switching between modes, and the bottom one to actually do the maths. The top breadboard has 26 LEDs on it, and will have 2 8-bit 2:1 multiplexers. The bottom one has 2 quad-XOR gates, 2 4-bit adders and 2 quad-NOR gates. The right shift is simply connecting the inputs to the output in a different place.

### The LEDs

This was a simple part. All I had to do was solder down a whole load of green and blue LEDs, some resistors and some wires. Despite this, I still came across a few issues when prototyping. The current-limiting resistors I used were 470R resistors, which pulled too much current and so the chips to actually use the data sometimes didn't have enough current to know if it was a 1 or a 0. This meant the built-in pull-up resistors took over and the actual data was ignored. I fixed this by using 10kR resistors instead.

## *Adding and Subtracting*

I used 4 chips for this, 2 adders and 2 XOR chips. Adding 2 numbers is simple: just feed them into the adders (I used 4-bit adders instead of making my own because it's cheaper). Subtracting is harder, as you have to negate the second number and then add them together. This is essentially just doing  $5 + -3$  instead of  $5 - 3$ . In binary, negative numbers are expressed in two's complement, which you convert into by inverting all of the bits and adding 1. I accomplished this by using a single subtract signal (this is actually the least significant bit of the setting as the setting for add is 00, so it won't subtract, subtract is 01 so it will subtract, and the other two don't matter as they won't use the adders anyway). The subtract signal is XORed with all of the data bits for the second number, as any number XOR 0 is itself, and any number XOR 1 is itself inverted. The subtract signal then also goes to the carry in of the first adder, so it will add one. This setup means that the second number will always go through the XOR gates, simplifying the wiring even if it does make the logic more complex.

---

## *NOR Operation*

This was a simple bit, as I just connected the inputs of the ALU to the inputs of the NOR gates. I came across no problems, and it was straightforward.

---

## *Right Shift*

I haven't done this yet, as it will be very simple. I don't have the multiplexers yet so I have to wait to connect the inputs to one.

---

## *Mode Selection*

So far, I have added the final 2:1 multiplexer, which will select between the adding/subtracting and the NOR-ing/right shifting. The select signal, outputs and A inputs have been wired up and connected. Next, I will add the multiplexers that will choose between NOR and right shift, but first I need to work out how to actually fit them on the breadboard!

---

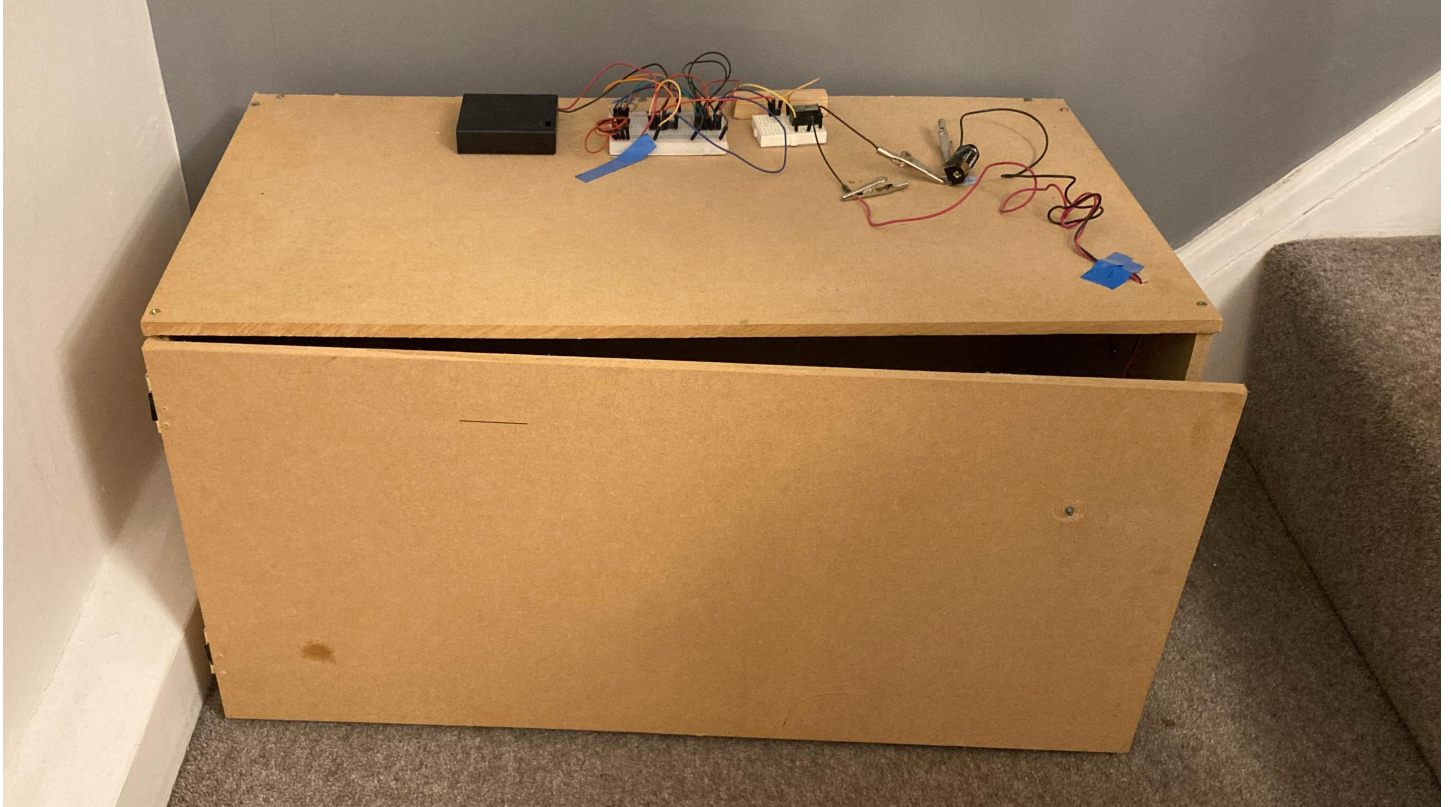
## *Challenges*

I didn't come across many challenges (yet) when making this. The main one was the weak resistors pulling all of the current so the chips registered a high input, but this was easily fixed. I still have the multiplexers to add though, so I may come across more problems.

## 4.2 Safe

---

2021 - Aged 10



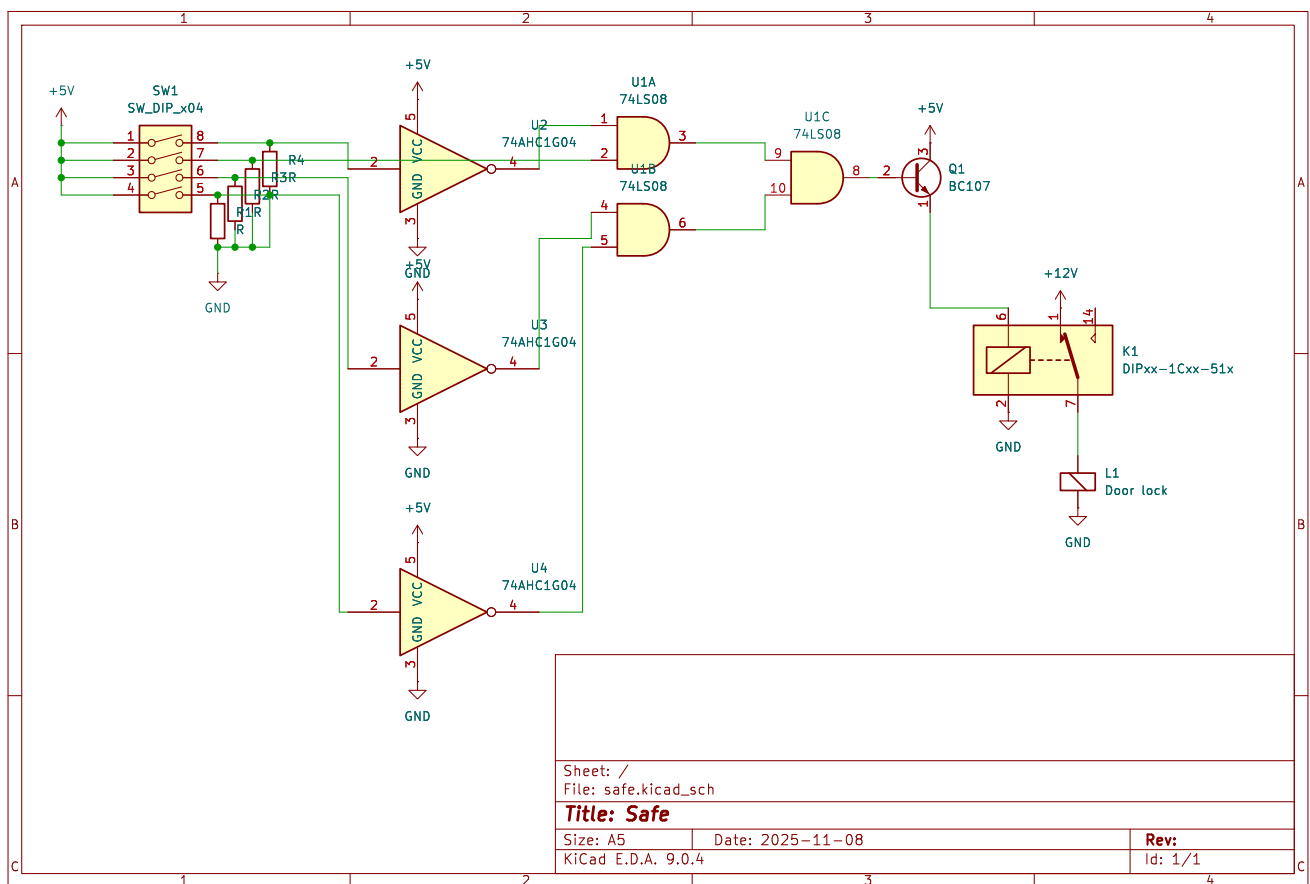
*My finished safe*

### 4.2.1 Overview

---

This is my first large electronics project. It isn't very complex by the standards of my other projects but it is on here because I had basically never done this sort of thing before. It is a homemade safe with a 4-bit code to unlock the electromagnet. I sawed the MDF sheets and screwed them together, before adding an electromagnet inside and some metal washers on the door. On top are the electronics, which consist of a 4.5V battery pack, a DIP (Dual Inline Package) switch, 4 AND gates, 6 NOT gates, a transistor, a relay and a 12V battery.

---



A schematic diagram of how the electronics work

## 4.2.2 My Aim

I wanted to make a safe with a door that locked, and you could only open it if you know the code. I mostly achieved that goal, as there is some resistance to the door when it is locked, but none when unlocked. However, it isn't a very safe safe as the electromagnet isn't very strong so you can just pull the door open, or you could just unplug the batteries.

## 4.2.3 How it works

1. There are 4 switches, each of which is on or off,
2. The 1st, 3rd and 4th switches are inverted,
3. The outputs of the inverters and the 2nd switch are ANDed together.  
This means that the output of this is only on if the 1st input is off, the 2nd input is on, and the 3rd and 4th inputs are off.
4. The output of this switches a transistor to increase current,
5. The output of the transistor switches a relay to increase voltage,
6. The relay switches the electromagnet,
7. The electromagnet pulls of washers glued to the inside of the door to close it

## 4.2.4 Challenges

---

It was my first large DIY project so I had to learn to use a saw and a drill safely, as well as the electronics. The main problem I had with the electronics was that I had forgotten pull-down resistors, so the inputs were floating (no definite value) when the switches were off. I eventually Googled the problem and put the resistors in and it worked, but I didn't realise why until I started making my CPU.

---

## 4.2.5 What I would change

---

If I built it again now, I would change a few things:

- I would use a solenoid pushing into a hole instead of an electromagnet.
- I would use a PCB to stop people unplugging wires.
- I would place all of the electronics inside the safe, with only the switches on top.
- I would use more switches, e.g. a 10-bit code

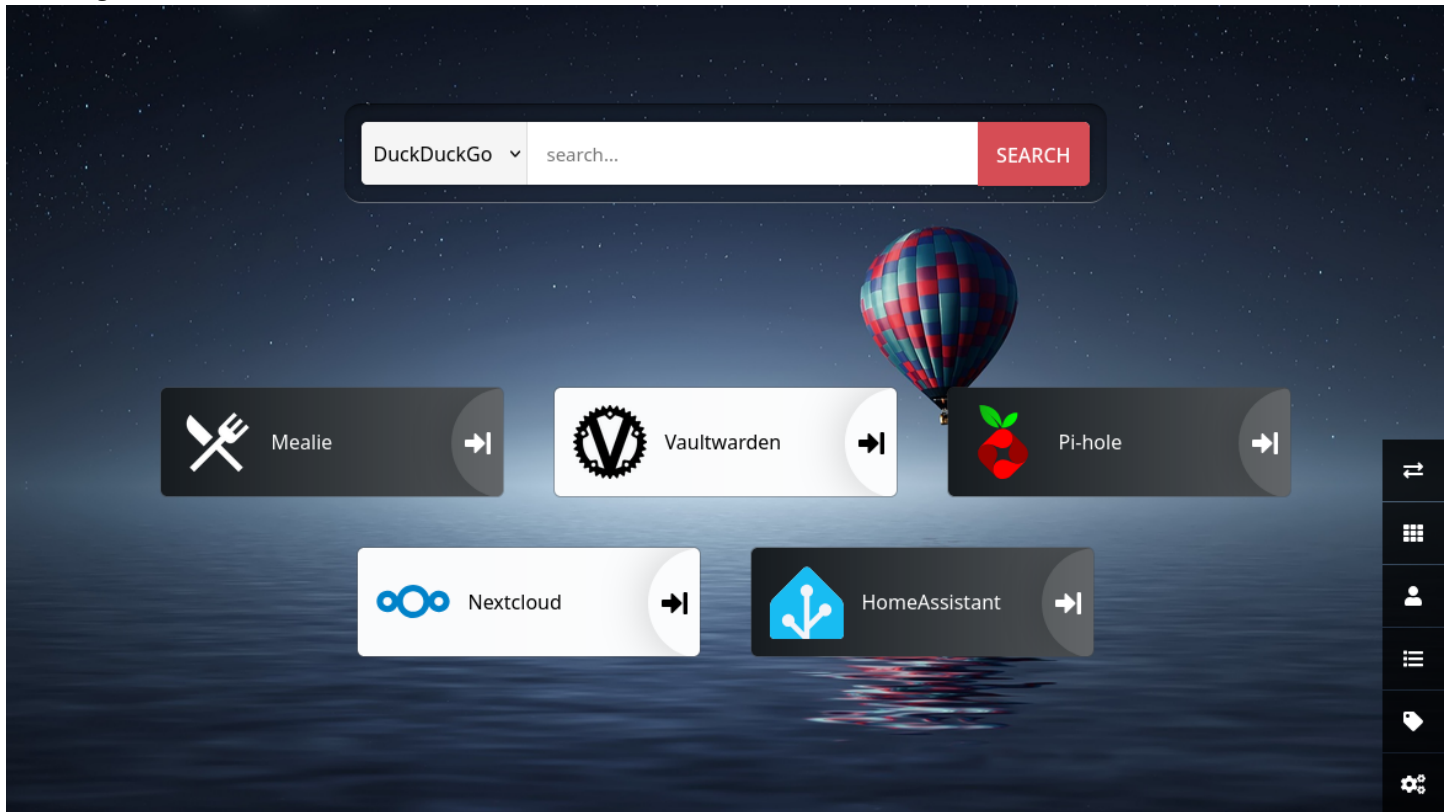
# 5. Digital Making

---

## 5.1 Home Server

---

2026 - Aged 15



*The dashboard*

### 5.1.1 Overview

This started as a small server to block ads on our network, then I gradually added more services. It currently runs Pi-Hole to block ads on our whole network, Mealie for a recipe website to help us decide what to eat, Vaultwarden to keep passwords safe and secure, Nextcloud for document sharing and editing, Heimdall for a dashboard and I am working on Mopidy, Home Assistant and PhotoPrism.

### 5.1.2 Hardware

I am currently using a Raspberry Pi 4 with a 64GB microSD card to host the server. This works for the services I have so far but apps like PhotoPrism are more demanding so I am considering upgrading. I have made a part list for the PC that I would upgrade to here (<https://uk.pcpartpicker.com/list/RVxgQy>). It doesn't yet include a case or any hard drives, but I am looking around, and have found the parts for much less than the advertised prices.

---

## 5.1.3 Architecture

The Raspberry Pi runs Debian and hosts the services. I gave it a static IP address with NetworkManager. Docker Compose manages the containerised applications, while Pi-hole provides DNS and DHCP for the network. Each service has its own local domain:

- `mealie.home.arpa`
- `warden.home.arpa`
- Other services use similar local domains.

Heimdall is a dashboard linking to the other services so I don't have to memorise the individual URLs. Most services store their persistent data in bind-mounted directories, allowing containers to be recreated without losing their configuration.

---

## 5.1.4 Pi-hole

Pi-hole (<https://pi-hole.net/>) is a network-wide ad blocker. You use it as your DNS server and any domains that serve ads are not returned. This means many ads are blocked. I first installed this with the install script, but I recently moved to managing it with Docker Compose to make it easier. Unfortunately my router (a Sky one) doesn't allow me to change its DNS server, so in order to make Pi-hole the default for the network I disabled router DHCP and enabled Pi-hole DHCP. I also set up custom domains, one for each other service (e.g. `mealie.home.arpa` for Mealie, `warden.home.arpa` for Vaultwarden).

---

## 5.1.5 Mealie

Mealie (<https://mealie.io/>) was the second service I installed. I used Docker Compose to install it. It is a recipe website, where I can add recipes, manage tags, categories, etc. It supports filtering by tags, categories, tools and ingredients used, as well as having a meal planner and shopping list. It was quite simple to set up, I just added about 15 lines to `compose.yml`.

---

## 5.1.6 Vaultwarden

Vaultwarden (<https://vaultwarden.com/>) is one of my most used services. It is a free, open source password manager fully compatible with Bitwarden. It was quite easy to set up (Docker Compose is really useful!) and it stores most of my passwords and other login information. It doesn't have its own client apps, but you can just use the Bitwarden browser extension and mobile app and it works fine.

---

## 5.1.7 Nextcloud

I don't use Nextcloud (<https://nextcloud.com/>) much right now, mostly because it doesn't run very well on my current hardware, but I still have it for when I need it. When combined with extensions, it is basically a Microsoft 365 replacement, with file storage, document editing, document sharing and similar features. I installed this a little differently to the rest, because I had to install a collection of libraries, unzip an archive in DocumentRoot and point `nginx` to it.

---

## 5.1.8 Mopidy

---

Mopidy (<https://mopidy.com/>) is a music server. It was designed for a single device, but I am going to use a piece of software called Snapcast to broadcast the music to other devices. Mopidy will find the music from local files, Spotify, TuneIn, etc. and Iris will provide the interface so I can decide what to play. Snapcast will then take the music and send it in frames to clients around the house so I can have fully synchronised music everywhere. Mopidy has no official Docker image, so I installed it with `apt` and most of the extensions with `pip3`. Snapcast also doesn't have an official Docker image so I will have to download the `.deb` file and manually install it.

---

## 5.1.9 Home Assistant

---

Home Assistant (<https://www.home-assistant.io/>) is a smart home manager. I installed it to see if we had anything that could be controlled but we don't, so I have temporarily disabled it.

---

## 5.1.10 Heimdall

---

Heimdall (<https://heimdall.site/>) is a simple dashboard. I installed it with Docker Compose again and it is my default site. It has links to the other services I have, and is easy to configure.

---

## 5.1.11 Tailscale

---

To access the server from outside my LAN (e.g. to get passwords on my phone) I use a service called Tailscale (<https://tailscale.com/>). I created an account, then installed the app on the server and my phone. I signed in with the same account on both devices and it acted as a VPN, giving each device a Tailnet IP address so I could access one from the other. I then configured the server to be the DNS server for the Tailnet, as well as being a subnet router so I could use its LAN IP address. This meant I could access the services from anywhere in the world without port forwarding.

---

## 5.1.12 Docker compose

---

I use Docker because it keeps the data in one place and I have more control over starting, stopping, updating and recreating the containers. Every week or so I run `docker compose pull` to check for updates, and if there are any I restart the updated containers.

▼ Show/Hide Full `compose.yml`

```

services:

mealie:
  image: ghcr.io/mealie-recipes/mealie:latest
  container_name: mealie
  ports:
    - "9927:9000"
  volumes:
    - ./mealie-data:/app/data
  environment:
    - ALLOW_SIGNUP=true
    - PUID=1001
    - PGID=1001
    - TZ=Europe/London
    - MAX_WORKERS=1
    - WEB_CONCURRENCY=1
    - BASE_URL=http://mealie.home.arpa
  restart: always

heimdall:
  image: linuxserver/heimdall:latest
  container_name: heimdall
  ports:
    - 9999:80
  environment:
    - TZ=Europe/London
  volumes:
    - ./heimdall-data:/config
  restart: always

bitwarden:
  image: vaultwarden/server:latest
  container_name: bitwarden
  restart: unless-stopped
  volumes:
    - ./bw-data:/data
  ports:
    - "127.0.0.1:8080:80"
    - "127.0.0.1:3012:3012"

# homeassistant:
#   container_name: homeassistant
#   image: "ghcr.io/home-assistant/home-assistant:stable"
#   volumes:
#     - ./ha-data:/config
#     - /etc/localtime:/etc/localtime:ro
#     - /run/dbus:/run/dbus:ro
#   restart: unless-stopped
#   privileged: true
#   network_mode: host
#   environment:
#     TZ: Europe/London

pihole:
  container_name: pihole
  image: pihole/pihole:latest
  network_mode: host
  environment:
    TZ: 'Europe/London'
  volumes:
    - ./pihole-data:/etc/pihole
  cap_add:
    # See https://docs.pi-hole.net/docker/configuration/#note-on-capabilities
    # Required if you are using Pi-hole as your DHCP server, else not needed
    - NET_ADMIN
    # Optional, if Pi-hole should get some more processing time

```

```
- SYS_NICE  
restart: unless-stopped
```

---

## 5.1.13 Challenges

---

- Disabling router DHCP because it wouldn't let me change its DNS server
- Running multiple services on limited hardware
- Deciding which services should run in containers rather than bare metal (directly on the OS)
- Configuring DNS, DHCP and static IP addressing so the services worked together

---

## 5.1.14 What I Learnt

---

- How to use Docker and Docker Compose
- How DNS and DHCP work
- How to manage sites and reverse proxies with Nginx
- How to self-host applications
- How to configure and maintain a Linux server

## 5.2 Laptop

2023-2025 - Aged 12-14



*The working mini laptop*

### 5.2.1 Overview

I made this fully working miniature laptop from scratch. It uses a Raspberry Pi 4, a PiTFT touchscreen display, a miniature keyboard and a 3D printed case. It also has a rechargeable battery, but I haven't added an on/off switch yet so it needs to be plugged in. So far, it only boots to CLI but I am confident I can get it working to desktop.

### 5.2.2 Electronics

The battery is wrapped in duct tape to stop any possible shorts, then put in between the Pi and the screen. The screen is mounted directly on the Pi's GPIO pins, and has a small breakout of some of the pins. However, I had to bend these so the battery fit in. I am using a PowerBoost 1000C to connect the battery to the computer. The battery's power and ground lines are soldered onto the powerboost. Another pair of wires goes from the powerboost to some of the pins on the PiTFT to power it and the Pi. I also have a small speaker soldered to an amp, but don't yet have the amp's power or data lines connected and I need to add the power switch so I can turn it on and off. The keyboard is just a Rii mini wireless keyboard (<https://thepihut.com/products/miniature-wireless-usb-keyboard-with-touchpad-1>). All I had to do was plug the dongle into the Pi and it worked.

## 5.2.3 Software

---

The Raspberry Pi has the standard Raspberry Pi OS installed. I have tried to install the display drivers and they half work. It does display to the screen and is the right way up, but it only shows the console. When I use framebuffer tools such as `fb1` and `pygame`, it displays to the screen, so I can make keyboard-based Pygame games. However, as there is no X server running the mouse doesn't work. When I have another display hooked up to the HDMI ports, then it and the PiTFT have the exact same output until I run `startx`, then the PiTFT freezes, and only the HDMI display works until I end the X session.

---

## 5.2.4 Case

---

The case is 3D printed, and I found all the parts online here (<https://www.thingiverse.com/thing:864547>). I didn't have to do any CAD. However, because of the resolution of my printer, the screw threads didn't print properly, so I am holding the back on with tape. Also, the Pi can't be screwed into the holes so I have to design and print something to keep it from falling out.

---

## 5.2.5 Challenges

---

There were a number of challenges in making this project, some of which I haven't solved yet. The first one was that I used a PowerBoost 1000 instead of a PowerBoost 1000C, so I had no Micro USB charging port. I didn't even realise this until quite a long way into the project, when I tried to charge the battery and found I couldn't. I had to buy a new one, desolder/cut off wires from the old one and resolder the new one. The next (current) challenge is figuring out how to mirror the HDMI output to `/dev/fb1` which is the screen's framebuffer.

## 5.3 Light Fantastic

---

2023-2025 - Aged 12-14



*The Light Fantastic after winning a game*

### 5.3.1 Overview

The Light Fantastic is a project from the book Raspberry Pi Projects for Dummies (<https://www.dummies.com/book/technology/computers/hardware/raspberry-pi/raspberry-pi-projects-for-dummies-281598/>). It is a 4x4 RGB illuminated button matrix, so 16 individually addressable RGB lights under 16 buttons. In the book, they run a ribbon cable to a raspberry pi outside the box, but as the Pico has now been released, I am using that inside the box. It is now mostly working, but there are still 2 buttons that don't register presses.

### 5.3.2 How Was It Built?

1. I took a pre-made PCB and altered it
2. I soldered a bunch of LEDs and diodes
3. I attached wires around the different ports
4. I attached the wires to the Raspberry Pi Pico
5. I programmed it
6. I put it in a case

---

## 5.3.3 PCB Altering

First of all, I took the Sparkfun button pad pcb (<https://www.sparkfun.com/button-pad-4x4-breakout-pcb.html>) (target="\_blank" rel="noopener"). This was designed for this sort of project. However, I was going to use Neopixels, not standard RGB LEDs, so it needed some modification. I cut a whole load of traces on the board to prevent short-circuits. I also accidentally cut a trace that I wasn't supposed to so I had to solder a wire across it.

---

## 5.3.4 LEDs and Diodes

This project needed 16 WS2812B LEDs for the illumination and 16 diodes to prevent crashing and possible damage when 2 or more buttons are pressed at once. I soldered the diodes first, but not very well as it was one of my first times soldering. I then did the LEDs, which was a bit better because I had got more experience in my break for a few months. They weren't much better, though, as the pins were much closer together so were much harder to do.

---

## 5.3.5 Wiring it Up

The first wires I did were the Din wire, through a resistor, to the first LED. I then did the Dout of the last LED of each row to the Din of the first LED on the next row. Next, I did the power and ground rails for the LEDs. I was then able to test them. I found that 2 of the Neopixels were just broken so we had to get some more, but after that it worked! I then had to do the 8 wires for the button matrix which went quite well.

---

## 5.3.6 Pico Connection

I didn't really feel confident soldering wires straight to the Pico, and I also might want to remove it and use it for something else later. My solution to this was to buy some 2.54mm pitch screw terminals and solder only the pins I needed. This worked really well and when I attached the wires to them they just worked.

---

## 5.3.7 Programming

I am programming in CircuitPython as I don't want to download a new firmware file for every update to the code. I made a main library that handles the actual interfacing with the board. It reveals functions to set LEDs to values based on either their index or xy coordinates, clear the LEDs, write changes, get buttons pressed, wait for a button to be pressed and get multiple button presses. I can then use that to make games for it. So far, I have only made a Lights Out game, where you have to turn all of the LEDs off. When you push a button, it inverts it and the 4 buttons it's touching. There are multiple levels, each one requiring more and more presses to win.

---

## 5.3.8 Case

It was quite fragile outside of a case, so I designed a case in TinkerCAD and 3D printed it. I made it in two parts so I could put the electronics inside, but I didn't think how to close it. Currently, the two parts are held together by tape.

---

## 5.3.9 Progression

---

This was one of the main defining projects for me, as it taught me a lot. When I started, I didn't really know how to solder or how electronics worked, and I could only program in simple Python. Now, I can solder quite well, have a deep understanding of how electric devices work, and I can program microcontrollers to do what I want.

---

## 5.3.10 Challenges

---

I did come across multiple challenges when making this. The first main one was when I cut the wrong PCB trace and had to solder a wire across it. Then the pins of the LEDs were too close for me to solder at that skill level, so I had to do other things for a long time before I was able to do it. I then had trouble working out how to attach the wires to the Pico, which I only answered in early 2025. Finally, in addition to games, I wanted it to be used as a sort of macro keyboard. This is why I used CircuitPython. When it turned on, it would try to connect to the computer to register itself as a keyboard. However, this mean it would crash when it isn't connected to a computer, for example if I just wanted to play a game on it. I couldn't debug this as it only crashed when it is *not* connected to a serial port. Currently, I am just adding games to it as this is the main aim of it anyway, but I will come back to this problem eventually. I reckon I will have to connect it to my Raspberry Pi via UART instead of USB to debug it.

## 5.4 Magic Mirror

---

2023-2025 - Aged 12-14



*The magic mirror (the text is hard to see)*

### 5.4.1 Overview

A magic mirror is a mirror that has text and possibly images displayed on it. They typically work with a 2-way mirror where you can see through it as well as the reflection, and then just placing a screen behind it. I got the inspiration to make one from the MagPi issue #93 (<https://magazine.raspberrypi.com/issues/93>), where they finished a guide on creating your own. This project involved learning a new programming language, 3D CAD, API calls and Linux setup.

### 5.4.2 Electronics

I chose a Raspberry Pi here because it made it simple to run a display, had a full Linux OS and I already knew how to use them. I got a Raspberry Pi Touch Display which was very much overkill as once the project's completed, you are never going to touch it (it'll be behind a mirror), but I already had one so decided to use it. I then just got a Raspberry Pi 3, connected it to the screen, and moved on to software.

## 5.4.3 Software

---

First, I got the MagicMirror<sup>2</sup> (<https://magicmirror.builders/>) software installed and set up. By default, it had a few useful modules installed, such as the time and the weather, but I also decided to change most of them. I removed the 'compliment' module, because the screen I used was too small, and then found a few other ones online that I liked. Once I had finished customising the display, the modules I had were:

- Clock (<https://docs.magicmirror.builders/modules/clock.html>)
- Weather (<https://docs.magicmirror.builders/modules/weather.html>)
- News Feed (<https://docs.magicmirror.builders/modules/newsfeed.html>)
- Facts (<https://github.com/alanshen111/MMM-Facts>)
- Weather Effects (<https://github.com/cgillinger/MMM-WeatherEffects>)
- Advent Calendar (<https://github.com/Jopyth/MMM-Advent>)

I left it at this stage for a few months, then I decided to go a step further. I learnt how to program in JavaScript, then discovered how to use the MagicMirror<sup>2</sup> API to create my own modules. The first one I made gave you a random Pokemon each day, similar to this one (<https://github.com/NolanKingdon/MMM-DailyPokemon>) but with less information. I made a few more small ones, but I haven't yet made a big one as the case problems put me off this project.

---

## 5.4.4 Case

Surprisingly, this was the bit that took the longest. I measured the precise dimensions of the glass and the screen, and designed a case that would fit it perfectly. I then 3D printed it, which took almost 24 hours to complete, but when I tested it, it didn't fit! I had designed it to fit the mirror so neatly that the printer squishing the plastic as it moved made it too narrow! This put me off 3D printing far more than it should, and I just put the mirror in a shoebox and forgot about it for a year. Recently (late 2025), I rediscovered it and made a new case for it using a simple trick I learnt when making my Light Fantastic, where you increase the capacity of the case by a few mm in each direction. I printed it again (in 2 parts as the first one failed near the end) and it worked really well.

---

## 5.4.5 Challenges

The most challenging part was making the case, as when I first designed it, I forgot to account for the printer's tolerance, so the mirror simply would not fit. At first, I wasn't sure why this happened, but I eventually measured the dimensions of the inside of the case and found that they were about a millimeter too small. In addition, creating my own modules for the mirror meant that I had to learn a whole new language first, as I had only touched JavaScript a few times before, and now I needed to learn DOM modelling, external API access, timeouts, status codes and a whole load more.

---

## 5.4.6 What I Learned

- How two-way mirrors work in display projects
- Setting up and configuring Linux-based systems
- Customising and extending existing open-source software
- Writing JavaScript modules using an existing API
- Designing 3D-printed parts with appropriate tolerances
- Learning from failed designs and improving them through iteration

## 5.5 Wildlife Camera

---

2022 - Aged 11



5.5.1 Details coming soon!

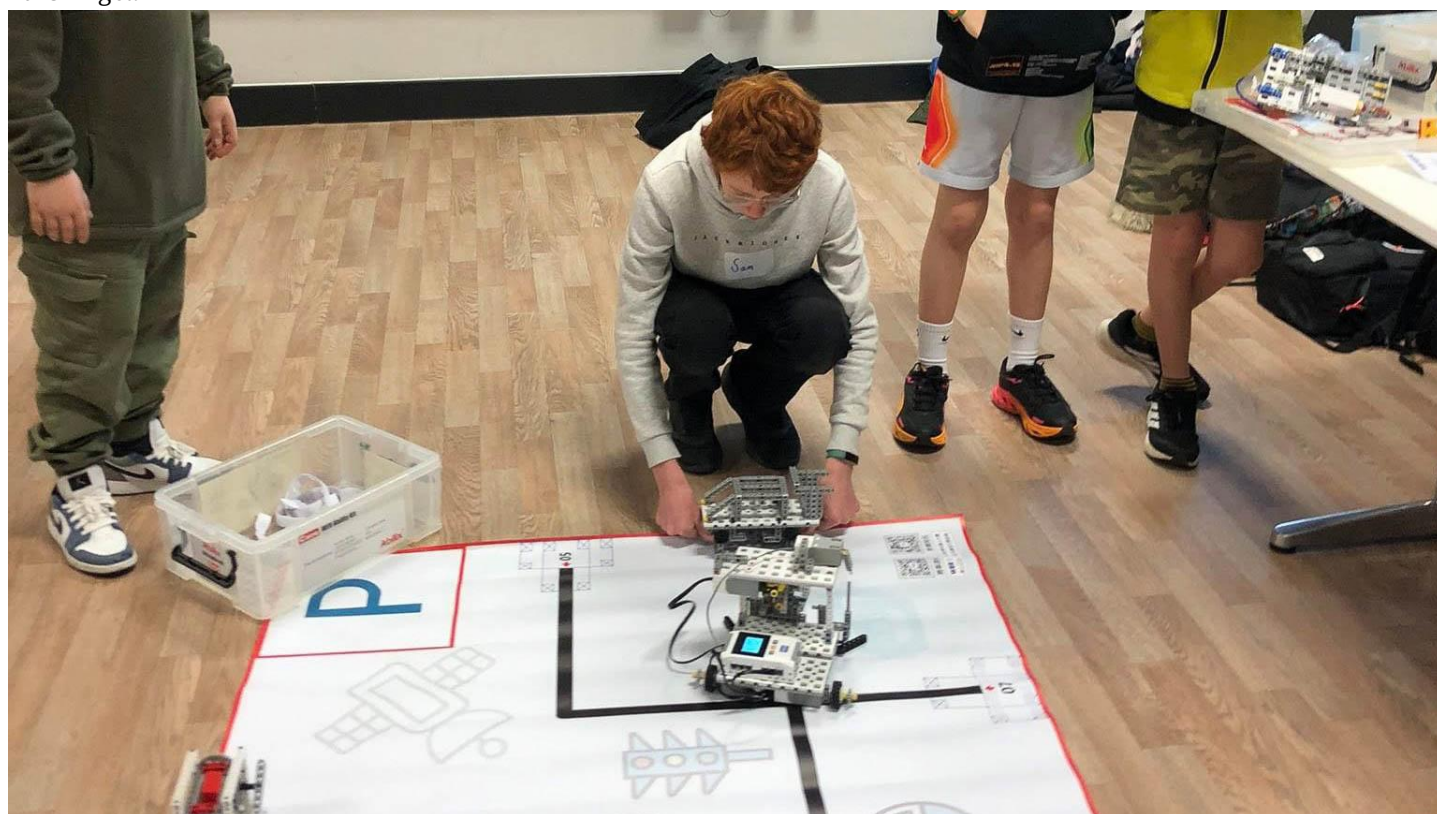
# 6. Robotics

---

## 6.1 World Educational Robotics 2025

---

2025 - Aged 14



*Me testing my robot on one of the tasks*

### 6.1.1 Overview

WER (<http://wergame.org/>) is an international educational robotics curriculum. There are many countries that host WER extra-curricular clubs with millions of participants, and some countries host robotics competitions. The most regular competitions take place in Britain every July and the winners from that competition qualify for the world championships in December in Shanghai. It is a large event, with over 6000 contestants in the championships (at least before Covid - there are fewer now). I joined in late December last year, and I went to the competition in England. I came 4th in my age category, but the top 3 all came from China to compete, so I qualified as the top British contestant and went to the world championships in Shanghai. The championships were on the 13th of December. It was a very stressful day, but in the end I got 3rd place!

### 6.1.2 About the competition

Each year, the competition has a different theme and map. This year was autonomous driving, and had pre-set tasks relating to the theme, e.g. Replacing the batteries in a model electric car and picking up containers to transport. You must compete in teams of 1 - 3. You get given a basic robot (plastic plate, 2 motors with wheels, 5 line sensors and a controller)

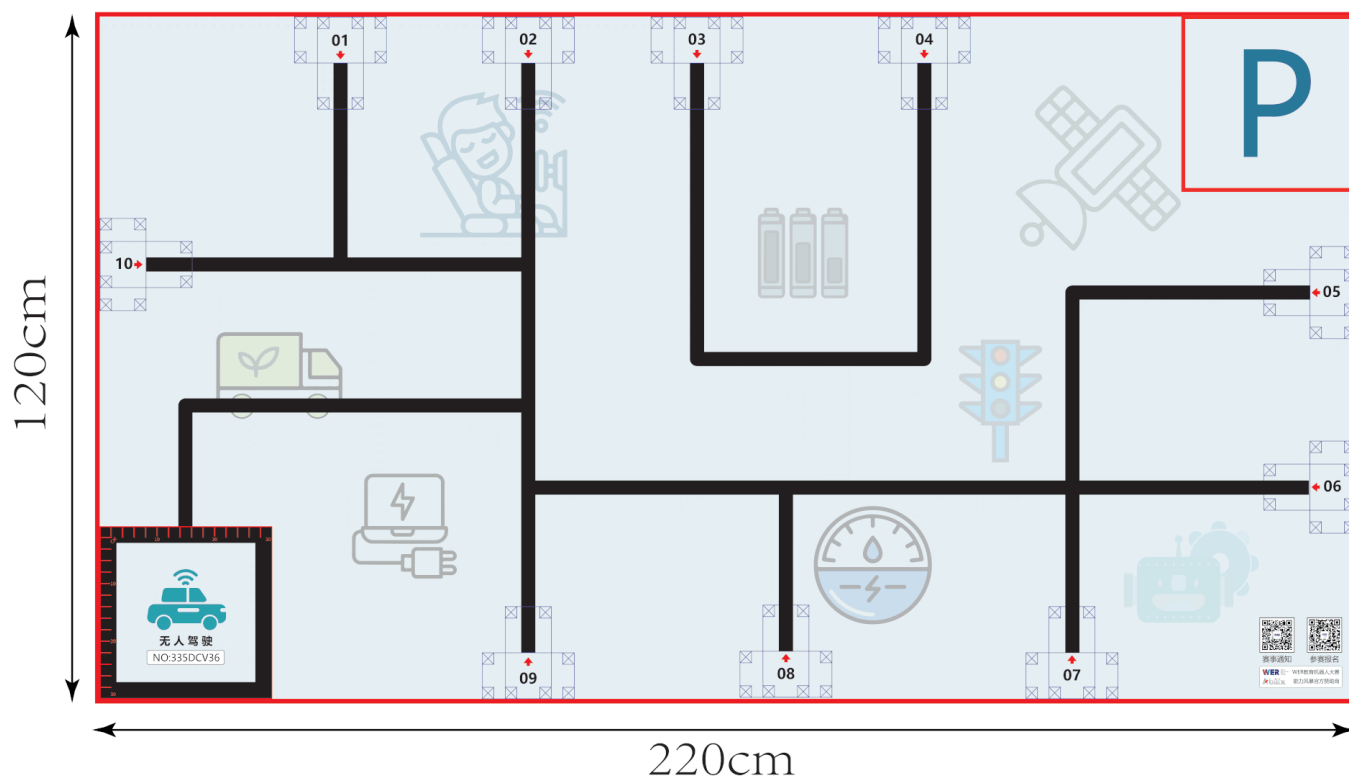
and must build additional things to complete the required tasks. It uses Abilix robots which have a meccano-style construction. You must also program the robot using either a version of Scratch or Python to move around the map and complete the required tasks.

## 6.1.3 This Year's Setup

The map was 120cm x 220cm, and it had a start box, 10 different positions and a parking space. The different ways of getting points were as follows:

- **Departure 20pts:** Leave the start box at least once.
- **Task 1 50pts:** Spin a bar around a certain number of times.
- **Task 2 90pts:**
  1. 10pts x 3: Remove 3 different coloured plastic blocks from a stand.
  2. 10pts x 3: Take all 3 blocks back to the start box.
  3. 30pts: Keep the blocks in the same order as they were on the stand.
- **Task 3 60pts:**
  1. 30pts: Spin a bar around to drop a red block on the ground.
  2. 30pts: Catch the red block and take it back to the start box.
- **Task 4 160pts:** This is a model of an electric car. It has a model of a dead battery in the bottom and a model of a charged battery in the top.
  1. 30pts: Remove the dead battery from the car.
  2. 30pts: Remove the charged battery from the car.
  3. 30pts: Take the dead battery back to the start box.
  4. 30pts: Take the charged battery back to the start box.
  5. 40pts: Put the charged battery back where the dead battery was.
- **Parking 60pts:**
  1. 30pts: End the run with at least 1 wheel in the parking space.
  2. 30pts: Have the red block from **Task 3** in the parking space at the end of your run.
- **Bonus points Up to 40pts:** You get 40 bonus points. Each time you have to pick the robot up outside the start box, you lose 10 of these bonus points, until you have none left.
- **Field tasks 100pts x 3:** There are 3 unseen tasks that first appear at the competition. Each one is worth 100pts.

Each of the tasks could be in any position. In case you're interested, the full rules are [here](#) and this was the map:



## 6.1.4 Saturday Classes

I go to a club every Saturday for an hour and a half. This is where I do most of my progress, testing, and learning. Here I get to use one of the club's robots. Before I went to the competition, about half of these sessions were learning things such as calibration curves and how to line up neatly with the tasks, and the other half were just preparing for the competition. Since the British competition, I have needed all the time I can get to train and practise, but now that the championships are over, I can relax again.

## 6.1.5 British Competition

When I got to the competition, I saw where the tasks were on the map and what the 2 new tasks were. I had 2 hours to get the robot to do all of the pre-set tasks in their locations and engineer solutions to the new tasks. Then we had a 3-minute judging session, lunch, and in the afternoon all of the tasks had been moved around. We had another 2 hours to prepare the robots and code for the new positions, then a 3 minute judging and the awards ceremony.



*I am being presented my award*

## 6.1.6 Home programing

As well as preparing my code at the Saturday classes, I also do some at home. The main bit that I did was a few months ago where I wrote all of the code to get to all of the positions and back in a single week, then on Saturday I tested it and most of it worked or almost worked first time! It only took me about 2 hours of debugging to get all of it working. I have also done other programming and preparing at home, sych as thinking of a few different methods for task 4. Ultimately, I didn't actually use any of those methods, as I saw some other really good designs at the competition which I have taken inspiration from.

## 6.1.7 World Championship

Shortly after we arrived at the venue, we saw the 3 new tasks:

- You had to lift a bar entirely off a stand and drop it on the floor.
- You had to spin something around (I can't remember the details)
- You had to lift a small plastic box up and place it on a higher platform.

In the opening ceremony, I had to read out the competitors oath (essentially saying that we won't cheat) in English. However, while I was waiting to go up on the stage, most other teams started writing code, testing and debugging early, so I lost about 40 minutes of preparation time compared to the other teams. I then managed to test everything, but I realised

that I had built my solution to task 4 the wrong way round! My code to go to the parking space also stopped working. In the end, I managed to get 350 points in the morning, which was the highest of the entire British division. In the afternoon, I managed to do a bit more. I solved the first of the additional tasks, and got the red block to the parking space (although neither of my wheels were actually in the box). I think I got 440 points. It was then the awards and closing ceremony. I came 3rd! I was aiming for top 50, so I was shocked when they read my name out!

---

## 6.1.8 My Code

---

I have around 320 lines of Python code, organised into 5 sections. The first section is the main function. This is what I change the most often. After that, I have the code to complete each task once in position. Next, I have utility functions, such as calibrating the sensors and leaving the start box. Section 4 contains the functions to take me to each position. Many of these have small speed optimizations, such as skipping lines instead of going round the longer way. Finally, I have the code to take the robot back to the start box from each position. You can see the code here (it isn't the most efficient, but it works well enough):

▼ Show/Hide Code

```

# IMPORTS
import robot

# MAIN
def main():
    start()
    park()

# UTILITY
def start():
    robot.WER_InitRobot(1,-1,2,1,1)
    robot.WER_SetMotor_T(30,30,1)
def setup():
    robot.WER_InitRobot(1,-1,2,1,1)
    robot.WER_Set_GrayscaleSensor()
def sideways():
    robot.WER_SetMotor_T(-30,-30,1)
    robot.WER_SetMotor_E(30,-30, 2, 2200)
    robot.WER_SetMotor_T(30,30,2)
    robot.WER_SetMotor_E(-30,30, 1, 2200)
    robot.WER_SetMotor_T(30,30,2.1)
    robot.WER_SetMotor_E(-30,30, 1, 2200)
    robot.WER_SetMotor_T(30,30,2.3)
def park():
    robot.WER_SetMotor_T(30, -30, 0.5)
    robot.WER_SetMotor_T(100, 100, 2.7)

# TASKS
def t1():
    robot.set_motor(3, -30)
    robot.sleep(1.2)
    robot.set_motor(3, 0)
def t2():
    robot.set_motor(3, 30)
    robot.sleep(1)
    robot.set_motor(3,10)
    robot.WER_SetMotor_T(-30, -30, 0.5)
    robot.set_motor(3, 0)
    robot.WER_SetMotor_T(-10, -10, 0.3)
def t3():
    robot.set_motor(3, 60)
    robot.sleep(1.5)
    robot.set_motor(3, 0)
def t4a():
    robot.set_motor(3, 10)
    while robot.get_encoder(3) < 1100:
        pass
    robot.set_motor(3,0)
    robot.set_motor(4,-30)
    robot.sleep(1)
    robot.set_motor(4,0)
def t4b():
    robot.set_motor(3,-10)
    while robot.get_encoder(3) > -1100:
        pass
    robot.set_motor(3,0)
def t5():
    robot.set_motor(3,20)
    robot.sleep(1)
    robot.set_motor(3,0)
def t6():
    pass
def t7():
    pass

```

```

# POSITIONS
def p1(task=None):
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(40,15,0.3)
    robot.WER_Around(-30,30,0)
    robot.WER_LineWay_C(40,1,0.3)
    robot.WER_Around(-30, 30,0)
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30, -30, 0)
    robot.WER_AdvancedWay_T(30,3)
    if task != None:
        task()
        r1()
def p2(task=None):
    robot.WER_LineWay_C(60,5,0.2)
    robot.WER_Around(50,-50,0)
    robot.WER_LineWay_C(60,15,0.2)
    robot.WER_Around(-50,50,0)
    robot.WER_LineWay_C(60,1,0.3)
    robot.WER_AdvancedWay_T(30,1.2)
    if task != None:
        task()
        r2()
def p3(task=None):
    robot.WER_LineWay_C(60,5,0.3)
    robot.WER_Around(60, -60,0)
    for _ in range(8):
        robot.WER_AdvancedWay_T(60,0.1)
    while robot.get_channel_gray(1,5) < 2000:
        robot.WER_SetMotor_K(40,40)
    robot.sleep(0.4)
    robot.WER_SetMotor_K(0,0)
    robot.WER_Around(-60, 60,0)
    robot.WER_LineWay_C(60,1,0.1)
    robot.WER_Around(-60,60,1)
    while(robot.get_channel_gray(1, 5) < 2000):
        robot.WER_SetMotor_K(-60,-60)
    robot.sleep(0.3)
    robot.WER_SetMotor_K(0,0)
    while(robot.get_channel_gray(1, 5) < 2000):
        robot.WER_SetMotor_K(-60,-60)
    robot.WER_SetMotor_K(0,0)
    robot.WER_SetMotor_T(60, 60, 0.2)
    robot.WER_Around(60, -60, 0)
    robot.WER_AdvancedWay_T(30,2)
    if task != None:
        task()
        r3()
def p4(task=None):
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(40,15,0.3)
    robot.WER_Around(-30,30,0)
    robot.WER_LineWay_C(40,1,0.3)
    while(robot.get_channel_gray(1, 5) < 1000):
        robot.WER_SetMotor_K(-35,35)
    robot.WER_SetMotor_K(0,0)
    while(robot.get_channel_gray(1, 3) < 1000):
        robot.WER_SetMotor_K(-35,35)
    robot.WER_SetMotor_K(0,0)
    while(robot.get_channel_gray(1, 5) < 1000):
        robot.WER_SetMotor_K(-35,-35)
    robot.sleep(0.3)
    robot.WER_SetMotor_K(0,0)
    while(robot.get_channel_gray(1, 5) < 1000):
        robot.WER_SetMotor_K(-35,-35)

```

```

robot.sleep(0.3)
robot.WER_SetMotor_K(0,0)
while(robot.get_channel_gray(1, 5) < 1000):
    robot.WER_SetMotor_K(-35,-37)
robot.WER_SetMotor_K(0,0)
robot.WER_SetMotor_T(40, 40, 0.3)
robot.WER_Around(30, -30, 0)
robot.WER_AdvancedWay_T(30,1.7)
if task != None:
    task()
    r4()
def p5(task=None):
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(40,15,0.3)
    robot.WER_SetMotor_T(40, 40, 0.3)
    while(robot.get_channel_gray(1, 5) < 2000):
        robot.WER_SetMotor_K(60,60)
    robot.sleep(0.3)
    robot.WER_SetMotor_K(0,0)
    robot.WER_Around(-30, 30, 0)
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_AdvancedWay_T(30, 2.2)

    if task != None:
        task()
        r5()
def p6(task=None):
    robot.WER_LineWay_C(60,5,0.2)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(60,15,0.2)
    robot.WER_Around(30, -30,0)
    robot.WER_LineWay_C(60,1,0.05)
    robot.WER_Around(-30, 30, 0)
    robot.WER_LineWay_C(60,5,0.2)
    robot.WER_SetMotor_T(30,30,0.2)
    robot.WER_LineWay_C(60,5,0.3)
    robot.WER_SetMotor_T(30,30,2.2)
    if task != None:
        task()
        r6()
def p7(task=None):
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(40,15,0.3)
    robot.WER_SetMotor_T(40, 40, 0.3)
    while(robot.get_channel_gray(1, 5) < 2000):
        robot.WER_SetMotor_K(60,60)
    robot.sleep(0.3)
    robot.WER_SetMotor_K(0,0)
    while robot.get_channel_gray(1,3) < 3000:
        robot.WER_SetMotor_K(40, -40)
    robot.WER_AdvancedWay_T(40,1.8)
    if task != None:
        task()
        r7()
def p8(task=None):
    robot.WER_LineWay_C(60,5,0.2)
    robot.WER_Around(50,-50,0)
    robot.WER_LineWay_C(60,15,0.2)
    x = robot.get_encoder(1)
    print(x)
    while robot.get_encoder(1) > x-5200:
        robot.WER_SetMotor_K(60,60)
        print(robot.get_encoder(1))
    robot.WER_SetMotor_K(0,0)

```

```

robot.WER_Around(60,0,0)
robot.WER_Around(60,0,0)
robot.WER_AdvancedWay_T(30, 1.2)

if task != None:
    task()
    r8()
def p9(task=None):
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(40,15,0.3)
    robot.WER_Around(30, -30,0)
    robot.WER_LineWay_C(30,1,0.3)
    robot.WER_AdvancedWay_T(20, 1.8)
    if task != None:
        task()
        r9()
def p10(task=None):
    robot.WER_LineWay_C(40,5,0.3)
    robot.WER_Around(30,-30,0)
    robot.WER_LineWay_C(40,15,0.3)
    robot.WER_Around(-30,30,0)
    robot.WER_LineWay_C(40,1,0.3)
    robot.WER_Around(-30, 30, 0)
    robot.WER_LineWay_C(40, 5, 0.3)
    robot.WER_AdvancedWay_T(30,1.2)
    if task != None:
        task()
        r10()

# RETURNS
def r1():
    robot.WER_SetMotor_T(-50, -60, 2.2)
def r2():
    while(robot.get_channel_gray(1, 1) < 2000):
        robot.WER_SetMotor_K(-60,-60)
    robot.sleep(0.2)
    while(robot.get_channel_gray(1, 1) < 2000):
        robot.WER_SetMotor_K(-60,-60)
    robot.WER_SetMotor_K(60, 60)
    robot.sleep(0.2)
    robot.WER_SetMotor_K(0,0)
    robot.WER_Around(-60, 60, 0)
    robot.WER_LineWay_C(60, 1, 0.2)
    robot.WER_Around(-60, 60, 0)
    robot.WER_LineWay_C(60, 15, 0.3)
def r3():
    while robot.get_channel_gray(1, 5) < 2000:
        robot.WER_SetMotor_K(-60, -60)
    robot.WER_SetMotor_T(-60, -60, 0.3)
    while robot.get_channel_gray(1, 5) < 2000:
        robot.WER_SetMotor_K(-60, -60)
    robot.WER_SetMotor_T(60,60,0.3)
    robot.WER_Around(-60, 60, 0)
    robot.WER_LineWay_C(60,15,0)
    robot.WER_SetMotor_T(60, 60, 0.3)
    while robot.get_channel_gray(1, 1) < 2000:
        robot.WER_SetMotor_K(60, 60)
    robot.WER_SetMotor_K(0,0)
    robot.WER_SetMotor_T(-60, 60, 0.4)
    robot.WER_SetMotor_T(60, 60, 0.3)
def r4():
    while robot.get_channel_gray(1, 1) < 2000:
        robot.WER_SetMotor_K(-30, -30)
    robot.WER_SetMotor_T(-30, -30, 0.3)
    while robot.get_channel_gray(1, 5) < 2000:
        robot.WER_SetMotor_K(-30, -30)

```

```

robot.WER_SetMotor_T(30,30,0.3)
robot.WER_Around(-30, 30, 0)
robot.WER_LineWay_C(40,15,0)
robot.WER_SetMotor_T(30, 30, 0.3)
while robot.get_channel_gray(1, 1) < 2000:
    robot.WER_SetMotor_K(30, 30)
robot.WER_SetMotor_K(0,0)
robot.WER_SetMotor_T(-30, 30, 0.5)
robot.WER_SetMotor_T(30, 30, 0.3)
def r5():
    robot.WER_SetMotor_T(-54,-50,5.5)
def r6():
    while robot.get_channel_gray(1,1) < 2000:
        robot.WER_SetMotor_K(-50,-50)
    while robot.get_channel_gray(1,1) >= 2000:
        robot.WER_SetMotor_K(-50,50)
    while robot.get_channel_gray(1,1) < 2000:
        robot.WER_SetMotor_K(-50,50)
    while robot.get_channel_gray(1,3) < 2000:
        robot.WER_SetMotor_K(-50,50)
    robot.WER_LineWay_C(60,15,0.3)
    while robot.get_channel_gray(1,1) < 2000:
        robot.WER_SetMotor_K(60,60)
    robot.sleep(0.2)
    robot.WER_SetMotor_T(-50,50,0.5)
    robot.WER_SetMotor_T(50,50,0.3)
def r7():
    while robot.get_channel_gray(1,1) < 2000:
        robot.WER_SetMotor_K(-40, -40)
    robot.WER_SetMotor_T(40,40,0.3)
    robot.WER_Around(40,-40,0)
    robot.WER_LineWay_C(50, 15,0.3)
    while robot.get_channel_gray(1,3) < 2000:
        robot.WER_SetMotor_K(40,40)
    robot.sleep(0.3)
    robot.WER_SetMotor_K(0,0)
    robot.WER_Around(-30,30,0)
    robot.WER_SetMotor_T(30,30,0.5)
def r8():
    robot.WER_SetMotor_T(-90,-60,4)
def r9():
    while robot.get_channel_gray(1, 1) < 2000:
        robot.WER_SetMotor_K(-30, -30)
    robot.WER_SetMotor_T(30, -30, 0.5)
    robot.WER_SetMotor_T(30, 28, 3)
def r10():
    while robot.get_channel_gray(1,5) < 2000:
        robot.WER_SetMotor_K(-30, -30)
    robot.WER_SetMotor_T(-30, 30, 0.3)
    robot.WER_SetMotor_T(30, 30, 3)

# START
main()#program entry

```

## 6.1.9 Challenges

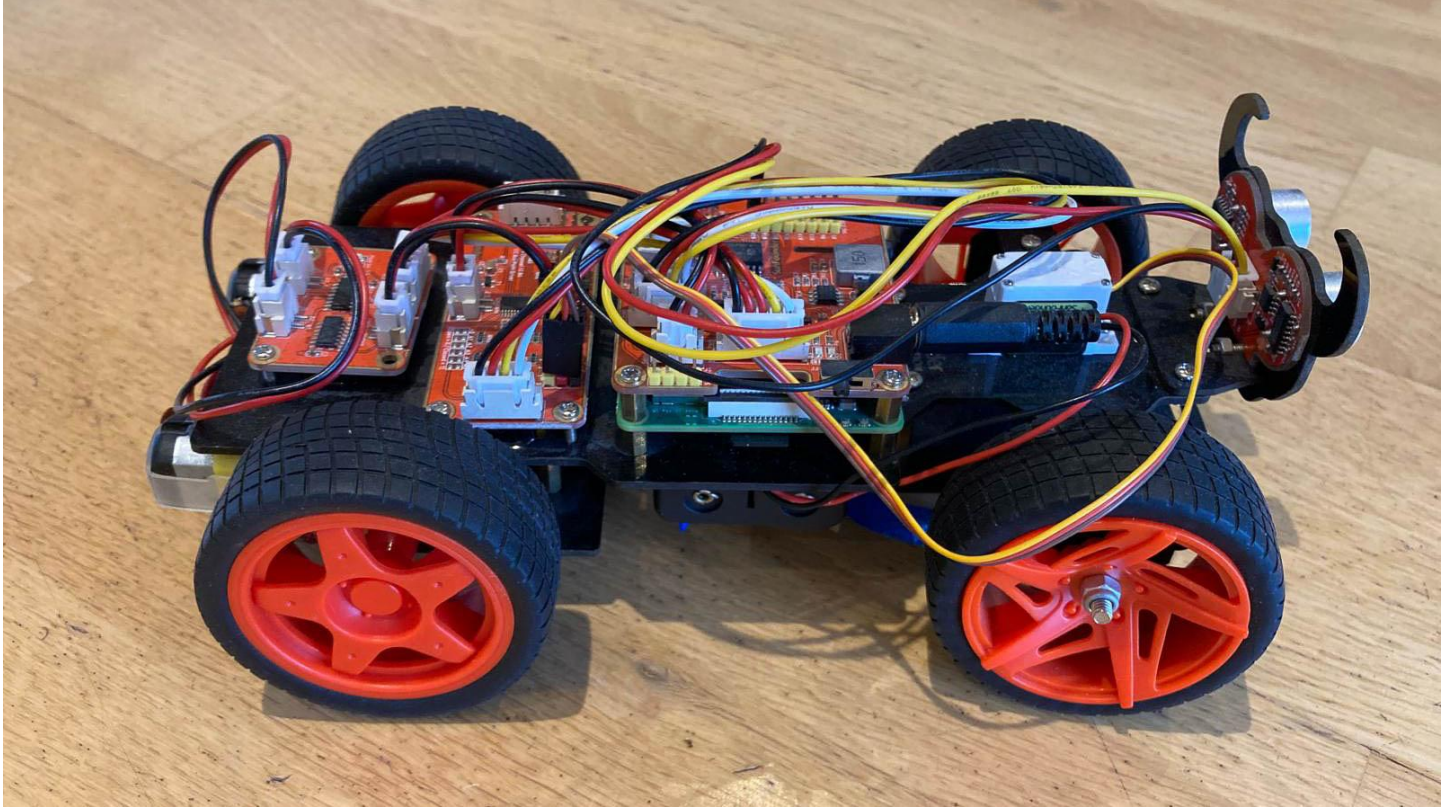
Doing this had a lot of challenges, from my first sessions building the robot, starting to program it, learning how to use C++ with the robot, etc. One of the biggest (unexpected) challenges was when they completely changed the syntax of the function calls 3 weeks before the competition! I had to spend a lot of time at home converting my 300 lines of code. Shortly after the competition, they changed from the old C202 robots to the new SK201 robots. This meant that I had to change all of

my code from C++ to Python. However, the functions were very similar so it didn't take too long. The most challenging task model was task 4, because the dead battery was too low for a motor to reach, so I had to use a crank linkage with a claw on the end to move it. Also, you had to put the battery back in the low position so the claw had to be able to grab and release.

## 6.2 Picar S

---

2023-2025 - Aged 12-14



### 6.2.1 Details Coming Soon!

---

## 6.3 My First Robot

---

2022 - Aged 11



### 6.3.1 Details coming soon!

---